

Seaborn과 시계열의 시작

한 줄로 그리고, 순서가 있는 데이터를 만납니다

histplot boxplot countplot barplot
scatterplot heatmap hue time_cycles

2026년 8월 26일 (수) · 26일차 · 63차시

교안 18_01 분포와 범주 시각화 · 18_02 관계 분석과 시각화 리포트 · 20_01 시계열 데이터의 이해

국립금오공과대학교 컴퓨터공학부 박민호

교안	다룬 범위	상태
18_01 분포와 범주 시각화	histplot · boxplot · countplot · barplot · 실습 1~8	완주
18_02 관계 분석과 리포트	scatterplot · corr · heatmap · subplots · 실습 1~6	완주
20_01 시계열 데이터의 이해	시계열 개념 · timestamp와 cycle · C-MAPSS · 실습 1~4	실습 4까지
20_01 실습 5~8 · 20_02	추세 그래프 · 시간 인덱스 · 리샘플링	다음 시간

이 책을 읽는 법

뱃지 · 코드블록 · 이전 회차와의 연결

1. 분류 뱃지

모든 항목 이름 앞에는 그것이 **무엇인지**를 알려주는 색 뱃지가 붙습니다. 오늘은 함수와 키워드가 특히 많이 나오는데, 이 둘을 구분하지 못하면 오류 메시지를 읽을 수 없습니다.

뱃지	무엇	판별 신호	오늘의 예
	라이브러리가 제공하는 함수	앞에 모듈 이름이 옵니다	<code>sns.histplot()</code> <code>sns.heatmap()</code>
	객체에 붙어 동작하는 함수	점 뒤에 있고 괄호가 있습니다	<code>df.corr()</code> <code>df.nunique()</code>
	함수에 넘기는 이름 붙은 인자	등호 왼쪽에 옵니다	<code>hue=</code> <code>bins=</code> <code>annot=</code>
	객체가 가진 값	점 뒤에 있고 괄호가 없습니다	<code>df.shape</code> <code>df.columns</code>
	문법이 아닌 사고 틀	코드로 쓸 수 없습니다	분포 · 상관 · 시계열 · 추세

개념

오늘 나오는 이름들의 **소속을 구별**해야 합니다. `sns.`로 시작하면 Seaborn 함수, `plt.`로 시작하면 Matplotlib 함수, `df.`로 시작하면 pandas 메서드입니다. **세 도구를 한 셀 안에서 섞어 쓰기 때문에**, 어떤 이름이 어디 소속인지 모르면 오류가 났을 때 어느 문서를 찾아야 할지도 알 수 없습니다.

2. 코드블록 읽는 법

회색 배경에 왼쪽 파란 바가 붙은 상자는 **파이썬 코드**입니다. 상자 아래 가로줄 밑에 초록색 ▶로 시작하는 줄은 **그 코드를 실제로 실행했을 때 나온 출력**입니다. 이 책의 모든 숫자는 실제 실행 결과입니다.

예시

```
import pandas as pd

df = pd.read_csv("Data/18_열처리.csv", encoding="utf-8")
print(df.shape)
print(df["판정"].value_counts())
```

- ▶ (297, 7)
- ▶ 판정
- ▶ 정상 285
- ▶ 이상 12
- ▶ Name: count, dtype: int64

주의

오늘 배우는 것도 어제처럼 **그림을 그리는 코드**라 출력의 숫자가 아니라 그래프입니다. 그래서 이 책은 **그래프 대신 그 그래프가 보여주는 내용을 숫자로 함께 실습니다**. 직접 그려 보시고 여기 실린 숫자와 대조해 보십시오. 그림과 숫자가 서로를 검증합니다.

3. 오늘 쓰는 데이터 두 가지

오늘은 **성격이 완전히 다른 두 데이터**를 다룹니다. 앞의 세 시간과 뒤의 두 시간이 나뉘는 지점입니다.

파일	크기	성격	쓰는 곳
18_열처리.csv	297행 7열	순서가 없는 스냅샷	STEP 1~7 (Seaborn)
20_cmapss_fd001_sample.csv	1116행 26열	순서가 있는 시계열	STEP 8~9 (시계열)

개념

앞의 데이터는 **행 순서를 섞어도 정보가 그대로**입니다. 297건의 측정을 아무 순서로 늘어봐도 평균·분포·상관은 변하지 않습니다. 뒤의 데이터는 **행 순서를 섞는 순간 정보가 사라집니다**. "점점 올라가고 있었다"는 사실이 순서 안에만 들어 있기 때문입니다. **이 차이가 오늘 후반부의 출발점**입니다.

4. 이전 회차와의 연결

어제는 Matplotlib으로 그래프의 부품을 하나씩 조립했습니다. 오늘은 **같은 그래프를 한 줄로** 그리고, 그다음 **새로운 종류의 데이터**로 넘어갑니다.



연결

어제 배운 것이 **오늘 그대로 다시 쓰입니다**. 히스토그램의 `bins`, 산점도의 `alpha`, `subplots`, `savefig`, `tight_layout` — 이름과 의미가 전부 같습니다. 달라지는 것은 **호출 방식**뿐입니다. 그래서 오늘 앞부분은 새로 외울 것이 적고, **어제 손으로 하던 일을 Seaborn이 대신해 주는 지점**을 확인하는 시간입니다.

구체적으로 어제 함수와 오늘 함수가 어떻게 대응되는지 미리 보고 가겠습니다. **왼쪽을 알면 오른쪽은 거의 자동으로 읽힙니다**.

어제 (25일차)	오늘 (26일차)	달라지는 점
<code>plt.hist(df["온도"])</code>	<code>sns.histplot(data=df, x="온도")</code>	축 이름이 자동으로 붙음
<code>plt.bar(x, y)</code>	<code>sns.countplot(data=df, x="라인")</code>	개수를 알아서 셈
<code>plt.bar(x, 평균)</code>	<code>sns.barplot(data=df, x="라인", y="온도")</code>	평균을 알아서 계산
<code>plt.scatter(x, y)</code>	<code>sns.scatterplot(data=df, x="A", y="B")</code>	같음
IQR 직접 계산	<code>sns.boxplot(data=df, y="온도")</code>	사분위수·수염·점 자동
<code>.map()</code>	<code>hue="판정"</code>	딕셔너리가 필요 없음
<code>plt.subplot(2,2,1)</code>	<code>fig, axs = plt.subplots(2,2)</code>	칸을 객체로 받음

주의

제목·축 이름·저장은 오늘도 plt 함수를 씁니다. Seaborn이 대체하는 것은 그래프 본체를 그리는 부분뿐입니다. `plt.title()` · `plt.xlabel()` · `plt.savefig()` · `plt.show()`는 어제 배운 그대로 쓰이므로, 어제 자료를 옆에 두고 보시면 좋습니다.

CONTENTS

목차

오늘 배우는 것을 한눈에

장	제목	무엇을 배우나
CH 0	오늘의 지도	Seaborn의 위치 · 그래프 고르는 두 기준
STEP 1	Seaborn 시작하기	import · set_theme · 한글 폰트 · sns와 plt의 분업
STEP 2	histplot으로 분포 보기	bins · kde · hue · 실습 1~4
STEP 3	boxplot으로 이상치 보기	사분위수 · 수염 · 범주별 비교 · 실습 5·6
STEP 4	범주형 데이터 세기	countplot · barplot · palette 규칙 · 실습 7·8
STEP 5	scatterplot으로 관계 보기	x·y 두 축 · alpha · hue로 위험 영역 · 실습 1·2
STEP 6	상관행렬과 heatmap	corr · annot · fmt · cmap · center · 실습 3
STEP 7	시각화 리포트 만들기	subplots · ax · savefig · 사실→의미→행동 · 실습 4~6
STEP 8	시계열 데이터란	순서의 의미 · timestamp와 cycle · 추세·변동성·이상
STEP 9	C-MAPSS 구조 읽기	엔진 5대 · 센서 21개 · 쓸 센서 고르기 · 실습 1~4
부록 A	치트시트	오늘 나온 모든 함수를 한 표에
부록 B	오류·함정 사전	노트북 코드 교정 · 자주 걸리는 함정
부록 C	실습 하이라이트와 다음 시간	핵심 결론 정리 · 20_01 실습 5~8 예고

개념

오늘의 한 문장. "코드가 짧아지면 더 많이 그려 보게 되고, 더 많이 그려 보면 더 많이 발견하게 된다." Seaborn의 가치는 예쁜 그림이 아니라 탐색 속도입니다. 한 줄로 그릴 수 있으면 열 개를 그려 보게 되고, 열 줄이 필요하다면 하나만 그리고 맙니다. 분석의 질은 그려 본 그래프의 개수에 비례합니다.

오늘의 지도

Seaborn의 위치와 그래프 고르는 기준

0-1. Matplotlib과 Seaborn의 관계

둘은 경쟁 관계가 아닙니다. **Seaborn은 Matplotlib 위에 얹혀 동작**하며, 자주 쓰는 조합을 미리 묶어 둔 도구입니다.

구분		
위치	바닥에 깔린 그리기 엔진	그 위에 얹은 편의 계층
별명	plt	sns
입력 방식	열을 직접 꺼내 넘김	data=df로 통째로 넘김
색 지정	하나하나 지정	기본값이 이미 보기 좋음
그룹 나누기	.map()으로 직접 만들	hue= 한 단어
통계 계산	직접 계산해서 넘김	평균·사분위수를 알아서 계산
제목·축·저장	plt.title() 등	Matplotlib 것을 그대로 씀

비유

Matplotlib이 **밀가루·계란·설탕**이라면 Seaborn은 **핫케이크 믹스**입니다. 믹스를 쓴다고 밀가루를 안 쓰는 것이 아니라, 이미 섞여 있을 뿐입니다. 그래서 **맛을 조절하려면 결국 원재료를 알아야** 합니다. 어제 Matplotlib을 먼저 배운 이유가 여기 있습니다.

0-2. 코드 한 줄로 보는 차이

같은 그림, 다른 코드

```
# 어제 — Matplotlib으로 판정별 색 구분
color_map = {"정상": "blue", "이상": "red"}
colors = df["판정"].map(color_map)
plt.scatter(df["소입로온도"], df["제어출력"], c=colors, alpha=0.6)
plt.legend() # 범례는 따로 만들어 줘야 함
plt.show()
```

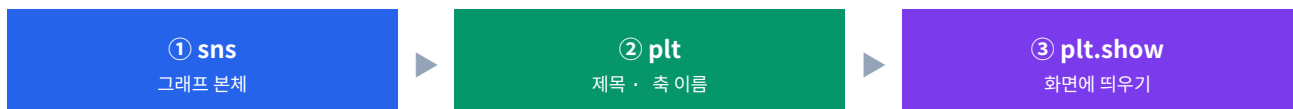
```
# 오늘 — Seaborn으로 같은 그림
sns.scatterplot(data=df, x="소입로온도", y="제어출력",
                hue="판정", alpha=0.6)
plt.show()
```

▶ 두 그림의 내용은 같습니다

▶ 다만 아래쪽은 색 지정·범례·정렬이 전부 자동입니다

줄 수는 여섯 줄에서 세 줄로 줄었지만, 실제 차이는 그보다 큼니다. **딕셔너리를 만들 필요가 없고, 색이 몇 개 필요한지 미리 알 필요도 없습니다.** 판정에 값이 세 종류로 늘어도 코드는 그대로입니다.

0-3. sns와 plt의 분업 — 오늘 모든 코드의 공통 골격



단계	누가	무엇을	예
①	sns	그래프 본체를 그림	sns.histplot(data=df, x="소입로온도")
②	plt	제목·축 이름·크기	plt.title("소입로온도 분포")
③	plt	화면 출력 또는 저장	plt.show()

개념

오늘 배우는 여섯 개 그래프가 전부 이 골격을 따릅니다. 바뀌는 것은 ① 단계의 함수 이름 하나뿐입니다. histplot 자리에 boxplot을 넣으면 박스플롯이 되고 heatmap을 넣으면 열지도가 됩니다. ②와 ③은 **어제 배운 그대로**입니다. 그래서 오늘 실제로 새로 외울 것은 **함수 이름 여섯 개와 옵션 몇 개**뿐입니다.

0-4. 그래프를 고르는 두 가지 질문

그래프 종류가 많아 보여도 **두 가지만 따지면** 후보가 하나로 좁혀집니다.

① 무엇을 보려는가	② 어떤 데이터인가	→ 그래프
분포 모양	숫자 1개	histplot
요약과 이상치	숫자 1개	boxplot
범주별 개수	범주 1개	countplot
범주별 평균	범주 1개 + 숫자 1개	barplot
두 변수의 관계	숫자 2개	scatterplot
여러 변수의 관계	숫자 여러 개	heatmap

주의

"라인별 소입로온도를 보고 싶다"처럼 막연하게 생각하면 그래프가 안 정해집니다. 라인별 **개수**인지, 라인별 **평균**인지, 라인별 **분포**인지를 먼저 정해야 합니다. 각각 countplot · barplot · boxplot으로 답이 달라집니다. 질문이 흐릿하면 그래프도 흐릿합니다.

0-5. 오늘 쓰는 열처리 데이터 살펴보기

실습 1 이전에 반드시 하는 일

```
import pandas as pd
import seaborn as sns
import matplotlib.pyplot as plt
```

```
df = pd.read_csv("Data/18_열처리.csv", encoding="utf-8")
print(df.shape)
df.info()
```

```

> (297, 7)
>
> RangeIndex: 297 entries, 0 to 296
> Data columns (total 7 columns):
>  0   라인    297 non-null  object
>  1   소입로온도  297 non-null  float64
>  2   제어출력   297 non-null  float64
>  3   건조출력   297 non-null  float64
>  4   세정기    297 non-null  float64
>  5   CP값     297 non-null  float64
>  6   판정     297 non-null  object
```

컬럼	종류	값의 범위 또는 종류	의미
라인	범주형	주간 100 · 야간 99 · 특근 98	근무조
소입로온도	숫자형	851.4 ~ 869.3 (평균 859.42)	소입로 온도(°C)
제어출력	숫자형	0.6 ~ 100.0 (평균 79.00)	히터 출력 비율(%)
건조출력	숫자형	10.9 ~ 29.0 (평균 22.16)	건조기 출력
세정기	숫자형	60.7 ~ 71.2 (평균 66.98)	세정기 온도
CP값	숫자형	0.400 ~ 0.607 (평균 0.450)	카본 포텐셜
판정	범주형	정상 285 · 이상 12	품질 판정

설비 적용

결측이 하나도 없습니다. info()에서 모든 열이 297 non-null이므로 22~23일차에 배운 결측 처리는 오늘 필요 없습니다. 대신 눈여겨볼 것은 **이상이 12건, 전체의 4.0%밖에 안 된다**는 점입니다. 이렇게 한쪽이 압도적으로 많은 것을 **불균형 데이터**라고 하고, 이런 데이터에서는 **평균 비교가 잘 통하지 않습니다**. 왜 그런지는 STEP 5에서 직접 확인하게 됩니다.

어제 쓰던 17_열처리.csv와 컬럼 이름이 조금 다릅니다. 어제는 배치(로트A·B·C)와 상태(정상·승온)였는데 오늘은 라인(주간·야간·특근)과 판정(정상·이상)입니다. 같은 공정을 다른 각도로 기록한 데이터라고 보시면 됩니다.

STEP 1

Seaborn 시작하기

import · set_theme · 한글 폰트

그리기 전에 **준비 세 줄**이 필요합니다. 이 세 줄은 그날 작업의 맨 위에 딱 한 번만 씁니다.

1-1. 세 도구를 함께 불러오기

문법 `import ... as ...` 라이브러리를 짧은 별명으로 불러오기

정의 `as` 뒤의 이름이 **별명**입니다. `pd` · `plt` · `sns`는 전 세계가 쓰는 공통 약속이라 임의로 바꾸지 않습니다.

형태 `import` 라이브러리 `as` 별명

```
import pandas as pd
import matplotlib.pyplot as plt
import seaborn as sns
```

▶ 출력은 없습니다 — 오류 없이 넘어가면 성공입니다

왜 쓰나 세 개를 다 불러와야 합니다. Seaborn만 있으면 제목도 못 달고 화면에 띄우지도 못합니다. `sns`는 그리기만 하고 나머지는 전부 `plt`가 하기 때문입니다.

응용 설치가 안 되어 있으면 `ModuleNotFoundError`가 납니다. 노트북 안에서 바로 설치할 수 있습니다.

```
# 주피터 노트북 셀에서 직접 설치
!pip install seaborn

# 버전 확인 — 옵션 규칙이 버전마다 다릅니다
print(sns.__version__)
```

▶ 0.13.2

개념

`import matplotlib.pyplot as plt`에서 **점 뒤의 pyplot**까지 써야 합니다. `import matplotlib as plt`로 쓰면 오류 없이 통과하지만 나중에 `plt.plot()`에서 `AttributeError`가 납니다. 그리기 함수들은 `matplotlib` 전체가 아니라 **그 안의 pyplot 모듈**에 들어 있기 때문입니다.

주의

별명을 바꾸지 마십시오. 인터넷의 모든 예제 코드가 `sns.plt.pd`를 쓰기 때문에, 별명을 바꾸면 **남의 코드를 복사해 올 때마다 고쳐야** 합니다.

1-2. 테마와 한글 폰트 — 순서가 중요합니다

함수 `sns.set_theme()` 그래프 전체 분위기를 한 번에 지정

정의 배경·격자·색 팔레트·글자 크기를 **한꺼번에** 정합니다. 그래프마다 반복할 필요가 없습니다.

형태 `sns.set_theme(style="whitegrid", context="talk")`

```
sns.set_theme(style="whitegrid")

# 한글 깨짐 방지 — set_theme 다음에 와야 합니다
plt.rcParams["font.family"] = "Malgun Gothic"
plt.rcParams["axes.unicode_minus"] = False
```

▶ 출력은 없습니다 — 이후 그리는 모든 그래프에 적용됩니다

왜 쓰나 매 그래프마다 색과 격자를 지정하는 수고를 없애 줍니다. **작업 시작 시 한 번만** 실행하면 그날 그리는 모든 그래프가 같은 스타일을 갖습니다.

응용 `style`은 배경 모양, `context`는 글자 크기를 정합니다. 발표용과 화면용의 글자 크기가 다릅니다.

```
# style — 배경과 격자
# "whitegrid"   흰 배경 + 회색 격자 (가장 무난)
# "darkgrid"   회색 배경 + 흰 격자 (기본값)
# "white"      격자 없는 흰 배경
# "ticks"      바깥 눈금선 추가

# context — 글자 크기
sns.set_theme(style="whitegrid", context="talk")
# "paper" < "notebook" (기본) < "talk" < "poster"
```

▶ `context="talk"`은 뒷자리에서도 읽히도록 글자를 키웁니다

개념

`set_theme()`이 `rcParams` 설정을 덮어씁니다. **그래서 순서가 중요합니다.** `set_theme()`을 먼저 하고 **그다음에** 폰트를 지정해야 합니다. 반대로 하면 한글이 다시 깨집니다. 오늘 실습 코드가 모두 이 순서인 이유입니다.

자주 하는 실수

폰트를 지정했는데도 깨지면 **글꼴 이름이 환경과 안 맞는 경우**입니다. 윈도우는 "Malgun Gothic", 맥은 "AppleGothic", 리눅스는 "NanumGothic"입니다.

1-3. 준비 코드 한 세트

노트북 첫 두 셀 — 복사해서 쓰십시오

```
import os
import pandas as pd
import seaborn as sns
import matplotlib.pyplot as plt

# 테마 먼저
sns.set_theme(style="whitegrid")

# 폰트는 그다음
plt.rcParams["font.family"] = "Malgun Gothic"
plt.rcParams["axes.unicode_minus"] = False

df = pd.read_csv("Data/18_열처리.csv", encoding="utf-8")
```

주의

axes.unicode_minus를 False로 두지 않으면 **음수 눈금의 마이너스가 네모로** 나옵니다. 오늘은 상관계수 heatmap에 **-0.74** 같은 음수가 표시되므로 반드시 필요합니다. 어제도 같은 이야기를 했는데, **매번 필요한 설정**이라 습관이 될 때까지 반복하는 것이 낫습니다.

1-4. 파일 경로 다루기

함수 `os.path.join()` 운영체제에 맞는 경로 만들기

정의 폴더 이름들을 받아 **그 운영체제에 맞는 구분자로** 이어 붙입니다. 윈도우는 역슬래시, 맥·리눅스는 슬래시를 씁니다.

형태 `os.path.join("../", "Data", "18_열처리.csv")`

```
import os

path = os.path.join("../", "Data", "18_열처리.csv")
print(path)

df = pd.read_csv(path, encoding="utf-8")
print(df.shape)
```

```
▶ ../Data\18_열처리.csv    (윈도우)
▶ ../Data/18_열처리.csv    (맥·리눅스)
▶ (297, 7)
```

왜 쓰나 노트북 파일이 Practice/ 안에 있고 데이터가 Data/ 안에 있으면 **한 단계 위로 올라갔다가 내려가야** 합니다.

..이 "한 단계 위"라는 뜻입니다.

응용 경로를 직접 문자열로 쓸 수도 있지만, 윈도우 역슬래시는 **탈출 문자**라 문제가 생깁니다.

```
# 위험 — \n, \t 등으로 해석될 수 있음
path = "../Data\\18_열처리.csv"    # 역슬래시 두 개

# 안전 1 — 슬래시를 쓰면 윈도우에서도 동작
path = "../Data/18_열처리.csv"

# 안전 2 — r을 붙여 원시 문자열로
path = r"../Data\18_열처리.csv"
```

개념

FileNotFoundError가 나면 **파일 이름 오타**이거나 **경로가 틀린 것** 둘 중 하나입니다. `print(os.getcwd())`로 현재 위치를 찍어 보고, `print(os.listdir())`로 그 폴더에 무엇이 있는지 확인하면 대부분 바로 해결됩니다.

주의

파이썬에서 **슬래시(/)**는 **윈도우에서도 동작합니다**. 역슬래시를 굳이 쓰지 않아도 되므로, 가장 간단한 해법은 그냥 슬래시로 쓰는 것입니다.

STEP 2

histplot으로 분포 보기

bins · kde · hue · 실습 1~4

분포는 **값들이 어느 구간에 얼마나 모여 있는지**입니다. 정상이 어떤 모양인지 알아야 이상을 알아볼 수 있으므로, **모든 분석은 분포 보기에서 시작합니다.**

2-1. 분포가 기준을 만든다

개념

진동값이 **평소 2 근처에 모인**다는 것을 알고 있으면, 4가 나왔을 때 "드문 값"이라고 판단할 수 있습니다. 그런데 평소 분포를 모르면 4가 큰 값인지 작은 값인지조차 알 수 없습니다.

분포를 안다는 것은 정상과 이상을 가르는 잣대를 갖는 것입니다. 이것이 분포를 가장 먼저 배우는 이유입니다.

2-2. 히스토그램이 만들어지는 원리

① 구간 나누기
같은 너비로

② 개수 세기
구간마다

③ 막대 높이
개수만큼

20일차에 배운 `pd.cut()`으로 구간을 나누고 `value_counts()`로 세던 일을 **한 줄로, 그림으로** 합니다. 원리를 알아야 그래프 모양도 정확히 읽힙니다.

구분		
보는 것	숫자 하나의 분포	서로 떨어진 범주의 크기
가로축	연속된 구간 (858~860°C)	독립된 범주 (주간·야간·특근)
막대 간격	붙어 있음	떨어져 있음
넣는 값	x= 숫자 컬럼 하나	x= 범주 컬럼 하나
함수	<code>sns.histplot()</code>	<code>sns.countplot()</code>

개념

막대가 붙어 있는지 떨어져 있는지가 눈으로 가르는 신호입니다. 히스토그램은 가로축이 끊김 없이 이어지므로 막대가 맞닿아 있고, 막대그래프는 범주 사이에 아무것도 없으므로 떨어져 있습니다. 어제 Matplotlib에서 배운 `plt.hist`와 `plt.bar`의 차이와 똑같은 이야기입니다.

2-3. 실습 1 — 첫 분포 그래프

함수 `sns.histplot()` 숫자 컬럼 하나의 분포를 막대로

정의 `data`에 데이터프레임을, `x`에 볼 컬럼 이름을 넣습니다. 열을 직접 꺼낼 필요가 없습니다.

형태 `sns.histplot(data=df, x="컬럼명", bins=구간수, kde=True, hue="범주컬럼")`

```
df_1 = pd.read_csv("Data/18_열처리.csv", encoding="utf-8")
```

```
# data와 x에 볼 컬럼을 지정
```

```
sns.histplot(data=df_1, x="소입로온도")
```

```
plt.title("소입로온도 분포")
```

```
plt.xlabel("소입로온도(°C)")
```

```
plt.show()
```

```
# 그림이 말하는 것을 숫자로 확인
```

```
print(df_1["소입로온도"].describe().round(3))
```

```
▶ count    297.000
▶ mean      859.423
▶ std        1.998
▶ min       851.400
▶ 25%       858.800
▶ 50%       859.700
▶ 75%       860.200
▶ max       869.300
```

왜 쓰나 어제 `plt.hist(df["온도"])`로 쓰던 것을 `sns.histplot(data=df, x="온도")`로 씁니다. 열을 꺼내는 대신 이름만 알려주면 되므로 컬럼을 바꿀 때 문자열 하나만 고치면 됩니다.

응용 축 이름이 컬럼 이름으로 자동으로 붙습니다. Matplotlib에서는 `xlabel`을 안 쓰면 축 이름이 아예 없었습니다.

```
# 이것만으로도 x축에 "소입로온도"가 자동으로 붙습니다
sns.histplot(data=df_1, x="소입로온도")
plt.show()

# 단위를 넣고 싶을 때만 직접 지정
sns.histplot(data=df_1, x="소입로온도")
plt.xlabel("소입로온도(°C)")
plt.show()
```

개념

x에 넣는 컬럼은 **반드시 숫자형**이어야 의미가 있습니다. 라인 같은 범주형을 넣어도 **오류는 안 나지만** 범주별 개수를 세는 다른 그래프가 되어 버립니다. 이 함정은 부록 B에서 실제 사례로 다룹니다.

주의

평균 **859.423**은 히스토그램의 봉우리 위치와 맞아야 하고, 표준편차 **1.998**은 봉우리의 폭과 맞아야 합니다. **그림을 그렸으면 describe()로 한 번 검사하는 습관을 들이십시오.**

2-4. 실습 2 — bins로 구간 수 조절

키워드 bins= 구간을 몇 개로 나눌지

정의 막대의 개수를 정합니다. **분포의 모양이 이 값에 따라 달라집니다.**

형태 sns.histplot(data=df, x="컬럼", bins=20)

```
import random

# bins를 여러 개로 바꿔 가며 그려 보기
bins_list = [random.randint(10, 30) for _ in range(5)]
print(bins_list)

for b in bins_list:
    plt.figure(figsize=(6, 4))
    sns.histplot(data=df_2, x="소입로온도", bins=b, kde=True)
    plt.title(f"Bins: {b}")
    plt.show()
```

- ▶ [13, 27, 18, 22, 11] (실행할 때마다 달라집니다)
- ▶
- ▶ bins가 작으면 뭉뚱그려지고, 크면 잘게 쪼개집니다

왜 쓰나 정답인 bins는 없습니다. 너무 작으면 봉우리 위치만 대략 보이고, 너무 크면 우연한 요철이 진짜 패턴처럼 보입니다. 여러 값으로 그려 보고 **일관되게 나타나는 특징만** 믿으면 됩니다.

응용 교안이 준 기준은 데이터 297건에 대해 **15~30 사이**입니다. 데이터 개수의 제곱근도 흔히 쓰는 출발점입니다.

```
# 제공근 기준 — 297건이면 약 17
import math
print(round(math.sqrt(len(df_2))))

# 구간 경계를 직접 정하고 싶을 때는 리스트로
sns.histplot(data=df_2, x="소입로온도",
              bins=[851, 855, 858, 861, 864, 870])
plt.show()
```

▶ 17

개념

random.randint(10, 30)은 **10 이상 30 이하**의 정수를 하나 뽑습니다. 끝값 30이 **포함된다**는 점이 range(10, 30)과 다릅니다. 실습 코드는 이렇게 뽑은 다섯 개를 반복문으로 돌려 **한눈에 비교**하는 구성입니다.

주의

반복문 안에서 plt.figure()를 **매번 새로** 만들어야 합니다. 빼먹으면 다섯 개의 히스토그램이 **한 그림 위에 겹쳐** 그려집니다.

키워드 kde=True 막대 위에 분포 곡선 그리기

정의 kde는 Kernel Density Estimate — 각 값 위에 작은 언덕을 올려 합친 **부드러운 추정 곡선**입니다.

형태 sns.histplot(data=df, x="컬럼", bins=20, kde=True)

```
sns.histplot(data=df_2, x="소입로온도", bins=20, kde=True)
plt.title("소입로온도 분포 (곡선 포함)")
plt.show()
```

- ▶ 막대 위에 매끄러운 곡선이 겹쳐 그려집니다
- ▶ 봉우리는 859~860°C 부근 하나입니다

왜 쓰나 막대는 bins에 따라 모양이 흔들리지만 **곡선은 상대적으로 안정적**입니다. 봉우리가 몇 개인지 파악할 때 특히 유용합니다.

응용 곡선의 봉우리가 둘이면 서로 다른 두 집단이 섞였을 가능성이 있습니다. 이럴 때는 hue로 갈라 보면 원인이 드러납니다.

```
# 봉우리가 둘로 보이면 그룹을 갈라 확인
sns.histplot(data=df_2, x="소입로온도",
              hue="라인", bins=20, kde=True)
plt.show()
```

▶ 오늘 데이터는 세 라인 모두 859~860°C 근처에 봉우리가 겹칩니다

개념

곡선은 추정입니다. 실제 값이 하나도 없는 구간에도 곡선이 이어질 수 있습니다. **정확한 개수는 반드시 막대로** 확인하십시오. 곡선은 모양을 보는 용도, 막대는 개수를 보는 용도입니다.

주의

kde=True만 쓰면 곡선의 세로축이 막대와 같은 개수 축에 맞춰집니다. 곡선만 보고 싶으면 `sns.kdeplot()` 이라는 전용 함수가 따로 있습니다.

2-5. 실습 3 — hue로 그룹별 분포 겹쳐 보기

키워드 hue= 범주 컬럼으로 색을 나눠 겹쳐 그리기

정의 hue에 넣은 컬럼의 값마다 **다른 색**으로 나누어 그림입니다. 색 지정도 범례도 자동입니다.

형태 `sns.histplot(data=df, x="숫자컬럼", hue="범주컬럼")`

```
df_3 = pd.read_csv("Data/18_열처리.csv", encoding="utf-8")

plt.figure(figsize=(10, 6))
sns.histplot(data=df_3, x="소입로온도", hue="판정", bins=20)
plt.title("양품 불량 분포 비교")
plt.show()

# 그림이 말하는 것을 숫자로
print(df_3.groupby("판정")["소입로온도"].agg(
    ["count", "min", "max", "mean", "std"]).round(2))
```

	count	min	max	mean	std
▶ 판정					
▶ 이상	12	851.4	869.3	856.88	6.91
▶ 정상	285	855.3	865.4	859.53	1.42

왜 쓰나 한 단어로 두 집단을 비교할 수 있습니다. Matplotlib에서 딕셔너리를 만들고 `.map()`으로 색을 매핑하던 작업이 통째로 사라집니다.

응용 이 그래프에서 가장 중요한 것은 **불량이 어느 쪽으로 치우쳤는가**입니다. 그런데 오늘 데이터는 답이 예상과 다릅니다.

```
# 이상 12건이 정상 온도 범위 밖에 있는지 확인
lo, hi = 855.3, 865.4      # 정상의 최소·최대
out = (df_3["소입로온도"] < lo) | (df_3["소입로온도"] > hi)

print("정상 띠 안:", (~out).sum(), "건 · 그중 이상", (~out &
(df_3["판정"]=="이상")).sum())
print("정상 띠 밖:", out.sum(), "건 · 그중 이상", (out & (df_3["판정"]=="이상")).sum())
```

▶ 정상 띠 안: 285 건 · 그중 이상 0

▶ 정상 띠 밖: 12 건 · 그중 이상 12

개념

이상의 평균 온도 856.88°C가 정상의 859.53°C보다 오히려 낮습니다. 평균만 보면 "온도가 낮으면 불량"이라는 이상한 결론이 나옵니다. 그런데 표준편차를 보면 이상이 6.91, 정상이 1.42로 약 4.9배 차이입니다. 즉 문제는 높낮이가 아니라 퍼짐입니다.

설비 적용

이상 12건이 정상 온도 범위 855.3~865.4°C 밖에 100% 존재합니다. 정상 띠 안 285건 중 이상은 단 한 건도 없습니다. 위험 영역은 "온도가 높은 곳"이 아니라 **정상 띠의 양쪽 바깥**입니다. 히스토그램에서 파란 봉우리 양옆으로 빠져나온 빨간 조각이 정확히 그것입니다.

자주 하는 실수

평균 비교로 그룹 차이를 판단하려다 결론을 놓칠 뻔한 사례입니다. 21일차에 배운 "평균이 같아도 퍼짐이 다르면 다른 공정"이 여기서 그대로 재현됩니다. hue 히스토그램이 좋은 이유는 **평균이 숨기는 것을 그림이 그대로 보여주기** 때문입니다.

2-6. 실습 4 — 여러 센서를 반복문으로

Practice/18_01_example.ipynb — 실습 4 (수정본)

```
df_4 = pd.read_csv("Data/18_열처리.csv", encoding="utf-8")

cols = list(df_4.columns)
print(cols)

# 숫자형 컬럼만 골라 차례로 그리기
for c in cols[1:-1]:
    plt.figure(figsize=(10, 6))
    sns.histplot(data=df_4, x=c, hue="판정", bins=30)
    plt.title(f"{c} - 판정별 분포")
    plt.show()
```

- ▶ ['라인', '소입로온도', '제어출력', '건조출력', '세정기', 'CP값', '판정']
- ▶
- ▶ 소입로온도 · 제어출력 · 건조출력 · 세정기 · CP값 다섯 장이 그려집니다

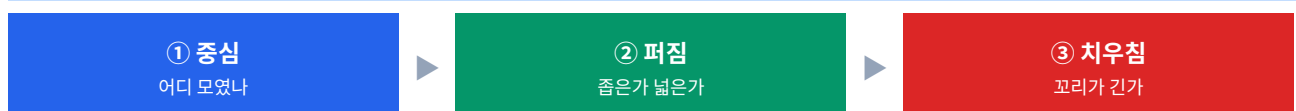
자주 하는 실수

제출된 노트북은 `for c in cols[0:-1]`로 되어 있습니다. 그런데 `cols[0]`은 라인, 즉 범주형입니다. 그래서 첫 번째 그림은 히스토그램이 아니라 라인별 개수 막대가 되어 버립니다.

오류가 나지 않으므로 알아채기 어렵습니다. 실제로 실행해 보면 막대가 6개(3라인 × 2판정) 나오는데, 이는 분포가 아니라 개수입니다. `cols[1:-1]`로 고쳐야 숫자형 다섯 개만 돌아갑니다.

반복문으로 다섯 장을 한꺼번에 그리는 것이 이 실습의 요령입니다. 컬럼 하나씩 손으로 쓰면 다섯 번 복사·붙여넣기를 해야 하고, 컬럼이 늘면 코드도 늘어납니다. 반복문을 쓰면 컬럼이 스무 개가 되어도 코드는 그대로입니다.

2-7. 분포를 읽는 세 가지 체크



체크	무엇을 보나	오늘 데이터	현장 의미
① 중심	막대가 가장 높은 구간	859~860°C	정상 작동 범위
② 퍼짐	좁게 모였나 넓게 흩어졌나	표준편차 1.998°C	설비 안정성
③ 치우침	한쪽 꼬리가 긴가	양쪽 모두 꼬리 존재	이상 후보

설비 적용

오늘 온도 분포는 **양쪽 꼬리가 모두 있습니다**. 최소 851.4°C와 최대 869.3°C가 평균 859.42°C에서 각각 8°C, 10°C 떨어져 있는데, 표준편차가 2°C이므로 **표준편차 네다섯 배**에 해당합니다. 한쪽만 길면 특정 방향의 문제지만, **양쪽이 다 길면 제어가 흔들린다**는 뜻입니다. STEP 5에서 그 이유가 밝혀집니다.

2-8. 센서별 정상 범위 메모하기

실습 4에서 다섯 장을 그렸으면 **끝이 아니라 시작**입니다. 각 센서가 어디에 몰려 있는지를 **숫자로 적어 두어야** 나중에 이상을 판정할 수 있습니다.

그림을 그렸으면 숫자로 남기십시오

다섯 센서의 정상 범위를 한 번에 정리

```
for c in df_4.select_dtypes("number").columns:
    m, s = df_4[c].mean(), df_4[c].std()
    q1, q3 = df_4[c].quantile([0.25, 0.75])
    print(f"{c:6} 평균 {m:8.3f} · 표준편차 {s:6.3f}"
          f" · 가운데 절반 {q1:.3f} ~ {q3:.3f}")
```

- ▶ 소입로온도 평균 859.423 · 표준편차 1.998 · 가운데 절반 858.800 ~ 860.200
- ▶ 제어출력 평균 79.004 · 표준편차 17.296 · 가운데 절반 72.900 ~ 90.400
- ▶ 건조출력 평균 22.158 · 표준편차 2.428 · 가운데 절반 20.800 ~ 23.500
- ▶ 세정기 평균 66.980 · 표준편차 1.636 · 가운데 절반 65.900 ~ 67.800
- ▶ CP값 평균 0.450 · 표준편차 0.010 · 가운데 절반 0.449 ~ 0.451

센서	평균	표준편차	가운데 절반 (Q1~Q3)	흔들림
소입로온도	859.423	1.998	858.8 ~ 860.2	0.23%
제어출력	79.004	17.296	72.9 ~ 90.4	21.89%
건조출력	22.158	2.428	20.8 ~ 23.5	10.96%
세정기	66.980	1.636	65.9 ~ 67.8	2.44%
CP값	0.450	0.010	0.449 ~ 0.451	2.20%

설비 적용

맨 오른쪽은 **표준편차를 평균으로 나눈 값**입니다. 크기가 다른 센서를 공평하게 비교하기 위한 계산으로, 어제 배운 것과 같은 발상입니다.

제어출력의 흔들림이 21.89%로 압도적으로 큼니다. 나머지는 전부 11% 이하입니다. 그런데 이것은 이상 신호가 아니라 **제어출력이 원래 그런 값**이기 때문입니다. 히터 출력은 0%에서 100%까지 계속 오르내리는 것이 정상입니다.

소입로온도의 흔들림은 0.23%로 가장 작습니다. 온도를 일정하게 유지하는 것이 이 설비의 목적이니 당연합니다. 즉 **제어출력이 크게 흔들리면서 온도를 좁게 유지하는 것이** 이 공정의 정상 상태입니다.

2-9. 어느 센서가 정상과 이상을 가르는가

실습 4에서 다섯 장을 그린 진짜 목적은 **어느 센서를 감시해야 하는지 고르는 것**입니다. 판정별로 갈라 숫자를 비교해 보겠습니다.

평균만 보지 말고 퍼짐도 함께 — 결과는 아래 표

```
# 센서마다 정상과 이상의 표준편차 배수를 비교
for c in df_4.select_dtypes("number").columns:
    s = df_4.groupby("판정")[c].std()
    print(f"{c:6} 표준편차 배수 {s['이상']/s['정상']:5.2f}")
```

센서	평균차	표준편차 배수 (이상/정상)	판단
소입로온도	2.655	4.86배	감시 대상 1순위
제어출력	4.025	2.82배	감시 대상 2순위
세정기	0.350	0.85배	차이 없음
건조출력	0.034	0.80배	차이 없음
CP값	0.002	0.46배	차이 없음

설비 적용

다섯 센서 중 둘만 의미가 있습니다. 소입로온도와 제어출력은 이상 그룹의 표준편차가 각각 4.86배, 2.82배로 훨씬 큼니다. 나머지 셋은 배수가 1보다 작아 **이상 그룹이 오히려 더 안정적**입니다 — 즉 판정과 아무 관계가 없습니다.

그리고 이 두 센서가 **바로 상관 -0.736으로 묶여 있는 그 쌍**입니다. 사실상 **하나의 제어 루프가 판정의 전부를 설명**하고 있는 셈이고, 여기서 STEP 5의 산점도로 이어집니다.

STEP 3

boxplot으로 이상치 보기

사분위수 · 수염 · 범주별 비교

히스토그램은 분포를 보여주지만 **정상 범위를 딱 잘라 말하지는 못합니다**. 박스플롯은 **분포 요약과 이상치 표시를 동시에** 합니다.

3-1. 박스플롯의 네 부분

부분	무엇	통계적 의미	오늘 소입로온도
상자	가운데 절반이 든 범위	Q1 ~ Q3 (IQR)	858.8 ~ 860.2
상자 안 가로선	데이터의 가운데 값	중앙값 (Q2)	859.7
수염	정상값의 끝	$Q1 - 1.5 \times IQR \sim Q3 + 1.5 \times IQR$	856.7 ~ 862.3
바깥 점	수염을 넘은 값	이상치 후보	32건

24일차에 손으로 계산하던 것을 boxplot이 대신합니다

박스플롯이 자동으로 계산하는 값을 직접 확인

```
q1 = df["소입로온도"].quantile(0.25)
```

```
q3 = df["소입로온도"].quantile(0.75)
```

```
iqr = q3 - q1
```

```
lo, hi = q1 - 1.5 * iqr, q3 + 1.5 * iqr
```

```
print(f"Q1 {q1} · Q3 {q3} · IQR {iqr:.3f}")
```

```
print(f"수염 경계 {lo:.3f} ~ {hi:.3f}")
```

```
m = (df["소입로온도"] < lo) | (df["소입로온도"] > hi)
```

```
print("바깥 점 개수:", m.sum())
```

▶ Q1 858.8 · Q3 860.2 · IQR 1.400

▶ 수염 경계 856.700 ~ 862.300

▶ 바깥 점 개수: 32

연결

24일차에 `quantile()`로 Q1·Q3을 구하고 IQR을 계산해 경계를 만들던 과정 전체를 `sns.boxplot()` 한 줄이 대신합니다. 그런데 원리를 모르면 그림을 읽을 수 없습니다. 상자 위아래가 왜 Q1·Q3인지, 수염이 왜 1.5배인지 모르면 점이 왜 찍혔는지도 설명하지 못합니다.

3-2. 중앙값이 평균보다 나은 경우

교안 예시를 그대로 실행

```
# 이상치가 섞였을 때 평균과 중앙값의 차이
vals = pd.Series([2.0, 2.1, 2.2, 2.3, 8.0])
print("평균 :", vals.mean())
print("중앙값:", vals.median())
```

▶ 평균 : 3.32

▶ 중앙값: 2.2

다섯 개 중 넷이 2 근처인데 **평균은 3.32**로 나옵니다. 8.0 하나가 평균을 끌어올린 것입니다. **중앙값 2.2**가 실제 대표값에 훨씬 가깝습니다.

설비 적용

설비 데이터는 **센서 오작동이나 순간 스파이크로 극단값이 섞이기 쉽습니다**. 그래서 대표값으로 평균보다 **중앙값이 더 믿을 만한** 경우가 많습니다. 박스플롯의 가로선이 평균이 아니라 중앙값인 것도 같은 이유입니다. 다만 중앙값은 **전체 크기 변화에 둔감**하므로, 실무에서는 둘을 함께 봅니다.

3-3. 실습 5 — 단일 센서 박스플롯

함수 `sns.boxplot()` 사분위수와 이상치를 상자로

정의 단일 분포를 볼 때는 y에 숫자 컬럼을 넣습니다. `histplot`이 x를 쓰는 것과 반대라 헷갈리기 쉽습니다.

형태 `sns.boxplot(data=df, y="숫자컬럼")` 또는 `sns.boxplot(data=df, x="범주", y="숫자")`

```
df_5 = pd.read_csv("Data/18_열처리.csv", encoding="utf-8")

plt.figure(figsize=(10, 6))
sns.boxplot(data=df_5, y="소입로온도")
plt.title("소입로 온도 IQR")
plt.ylabel("소입로온도(°C)")
plt.show()
```

▶ 세로 상자 하나와 위아래 수염, 그리고 바깥 점 32개

왜 쓰나 사분위수를 직접 계산할 필요가 없습니다. 상자·중앙값선·수염·이상치 점까지 전부 자동입니다. 24일차의 IQR 계산 코드 여섯 줄이 한 줄이 됩니다.

응용 다섯 센서를 한꺼번에 비교하려면 **컬럼을 세로로 편** 다음 넘기면 됩니다.

```
# 센서별 이상치 개수를 한 번에 세기
for c in ["소입로온도", "제어출력", "건조출력", "세정기", "CP값"]:
    a, b = df_5[c].quantile([0.25, 0.75])
    i = b - a
    k = ((df_5[c] < a - 1.5*i) | (df_5[c] > b + 1.5*i)).sum()
    print(f"{c:6} IQR {i:7.3f} · 이상치 {k:3}건")
```

▶ 소입로온도 IQR 1.400 · 이상치 32건
 ▶ 제어출력 IQR 17.500 · 이상치 16건
 ▶ 건조출력 IQR 2.700 · 이상치 11건
 ▶ 세정기 IQR 1.900 · 이상치 14건
 ▶ CP값 IQR 0.002 · 이상치 28건

개념

IQR이 좁을수록 이상치가 많이 잡힙니다. 소입로온도는 IQR이 1.4°C로 매우 좁아 297건 중 32건(10.8%)이 바깥 점으로 찍힙니다. CP값은 IQR이 0.002로 더 좁아 28건이 잡힙니다. 반면 제어출력은 IQR이 17.5로 넓어 16건뿐입니다.

자주 하는 실수

점이 많다고 다 이상은 아닙니다. IQR 기준은 "가운데 절반 대비 얼마나 먼가"를 볼 뿐이라, **원래 값이 촘촘히 모인** 센서일수록 점이 많이 찍힙니다. 소입로온도의 32건 중 실제 불량 판정은 12건뿐입니다. 박스플롯은 후보를 짚어 줄 뿐 판정하지 않습니다.

3-4. 실습 6 — 범주별 박스플롯

패턴 x=범주 · y=숫자 그룹별 상자를 나란히 비교

정의 x에 범주 컬럼, y에 숫자 컬럼을 넣으면 **범주 개수만큼 상자**가 나란히 그려집니다.

형태 `sns.boxplot(data=df, x="라인", y="소입로온도", hue="판정")`

```
df_6 = pd.read_csv("Data/18_열처리.csv", encoding="utf-8")

# 라인별 상자를 나란히
plt.figure(figsize=(10, 6))
sns.boxplot(data=df_6, x="라인", y="소입로온도")
plt.title("라인별 소입로온도 분포")
plt.show()

print(df_6.groupby("라인")["소입로온도"].describe().round(2))
```

	count	mean	std	min	25%	50%	75%	max
▶ 라인								
▶ 야간	99.0	859.64	1.93	851.4	859.10	859.9	860.15	868.7
▶ 주간	100.0	859.20	2.39	852.3	858.48	859.5	860.12	869.3
▶ 특근	98.0	859.43	1.57	851.9	858.82	859.8	860.28	863.4

왜 쓰나 어느 그룹을 먼저 봐야 할지 우선순위를 정할 수 있습니다. 세 상자를 나란히 놓으면 중앙값 높이와 상자 크기, 바깥 점 개수가 한눈에 비교됩니다.

응용 hue를 더하면 두 기준을 동시에 비교할 수 있습니다. 라인마다 정상·이상 상자가 두 개씩 나란히 그려집니다.

```
# 라인 x 판정 이중 비교
plt.figure(figsize=(10, 6))
sns.boxplot(data=df_6, x="라인", y="소입로온도", hue="판정")
plt.title("라인 x 판정 소입로온도")
plt.show()
```

- ▶ 라인마다 정상 상자와 이상 상자가 두 개씩
- ▶ 이상 상자는 세 라인 모두 위아래로 훨씬 길다

개념

평균은 세 라인이 859.20 · 859.43 · 859.64로 거의 같습니다. 그런데 표준편차는 특근 1.57, 야간 1.93, 주간 2.39로 주간이 가장 큼니다. 최댓값도 주간 869.3°C로 가장 높고 최솟값도 852.3°C로 낮습니다. **평균으로는 안 보이던 차이가 상자 크기로 드러납니다.**

설비 적용

주간 라인의 온도가 가장 많이 흔들립니다. 상자가 가장 크고 수염도 가장 길게 나옵니다. 실제로 불량률도 주간이 7.0%로 가장 높습니다(STEP 4에서 확인). **변동이 큰 라인에서 불량률이 많다**는 것은 예지보전에서 가장 기본적인 신호 중 하나입니다.

3-5. 일관된 패턴만 믿기

개념

hue까지 넣으면 상자가 여섯 개가 됩니다. 이때 **한두 군데서만 보이는 패턴은 우연일 수 있고, 모든 그룹에서 반복되는 패턴이 진짜 신호**입니다.

오늘 데이터에서는 **세 라인 모두** 이상 상자가 정상 상자보다 위아래로 길게 나옵니다. 라인과 무관하게 반복되므로 **믿을 만한 신호**입니다. 반대로 주간에서만 뭔가 보였다면 주간의 특수 사정일 뿐 공정 전체의 신호는 아닙니다.

자주 하는 실수

제출된 노트북 실습 6에는 `day = df_6[df_6["라인"] == "주간"]`으로 주간만 걸러 놓고, 정작 그릴 때는 `data=df_6(전체)`를 넘긴 코드가 있습니다. 제목은 "주간 라인"인데 **그림은 세 라인 전부**입니다.

오류가 나지 않으므로 알아채기 어렵습니다. **필터한 변수를 만들었으면 반드시 그 변수를 쓰는지 확인**하십시오. 아니면 아예 필터 줄을 지우고 제목을 고치는 편이 낫습니다.

STEP 4

범주형 데이터 세기

countplot · barplot · palette 규칙

숫자형은 **분포**로 보지만, 범주형은 **개수** 또는 **그룹별 값**으로 봅니다. "주간이 야간보다 크다"는 말은 성립하지 않기 때문입니다.

4-1. countplot과 barplot의 차이

구분		
보는 것	범주별 개수	범주별 평균
필요한 값	x만	x와 y 둘 다
막대 높이	몇 건인가	평균값이 얼마인가
질문	"주간에 데이터가 몇 개인가"	"주간의 평균 세정기 온도가 얼마인가"
오차선	없음	있음 (평균의 신뢰 구간)

주의

생김새가 똑같은 막대라 결과만 보고는 구분할 수 없습니다. "라인별 진동"이라고 막연히 생각하고 countplot을 쓰면 진동값은 무시되고 개수만 세어집니다. y가 필요한가를 따지는 것이 가장 확실한 구분 기준입니다.

4-2. 실습 7 — countplot과 barplot

함수 `sns.countplot()` 범주별 개수를 막대로

정의 x에 범주 컬럼만 넣으면 개수를 자동 집계해서 막대로 그립니다. `value_counts()`의 그림 버전입니다.

형태 `sns.countplot(data=df, x="범주컬럼", hue="범주컬럼", palette="Set2", legend=False)`

```
df_7 = pd.read_csv("Data/18_열처리.csv", encoding="utf-8")

sns.countplot(data=df_7, x="라인", hue="라인",
               palette="Set2", legend=False)
plt.title("라인별 데이터 개수")
plt.show()

print(df_7["라인"].value_counts())
```

```
▶ 라인
▶ 주간 100
▶ 야간 99
▶ 특근 98
▶ Name: count, dtype: int64
```

왜 쓰나 데이터 불균형을 즉시 확인할 수 있습니다. 라인별로 데이터 개수가 크게 다르면 이후 비교가 공정하지 않으므로, 분석 초반에 반드시 확인해야 합니다.

응용 hue를 더하면 범주 안의 범주까지 색으로 나눕니다. 오늘 실습 8이 이 방식입니다.

```
# 라인 안에서 정상·이상 개수까지
plt.figure(figsize=(10, 6))
sns.countplot(data=df_7, x="라인", hue="판정", palette="Set2")
plt.title("라인별 양품 / 불량 개수")
plt.xlabel("라인")
plt.ylabel("개수")
plt.show()
```

```
▶ 라인마다 정상 막대와 이상 막대가 두 개씩
▶ 이상 막대는 정상 막대에 비해 아주 작습니다
```

개념

라인별 개수가 **100 · 99 · 98**로 거의 같습니다. 이렇게 균등하면 개수 비교와 비율 비교의 결론이 같아집니다.

하지만 항상 그렇지는 않습니다. 개수가 크게 다르면 개수만 보고 판단하면 안 되고 반드시 비율을 계산해야 합니다.

주의

countplot은 y를 주지 않습니다. y를 주면 방향이 가로로 바뀌거나 오류가 납니다. **개수를 세는 그래프에는 셀 값이 필요 없다**고 기억하십시오.

함수 `sns.barplot()` 범주별 평균을 막대로

정의 x에 범주, y에 숫자를 넣으면 **범주별 평균**을 막대 높이로 그립니다. 합계나 중앙값이 아니라 평균이 기본입니다.

형태 `sns.barplot(data=df, x="범주", y="숫자", estimator="mean")`

```
# 라인별 평균 세정기 온도
sns.barplot(data=df_7, x="라인", y="세정기",
            hue="라인", palette="pastel", legend=False)
plt.title("라인별 평균 세정기 온도")
plt.show()
```

```
print(df_7.groupby("라인")["세정기"].mean().round(3))
```

```
▶ 라인
▶ 야간  66.281
▶ 주간  67.033
▶ 특근  67.634
▶ Name: 세정기, dtype: float64
```

왜 쓰나 21일차에 배운 `groupby().mean()`의 그림 버전입니다. **평균을 직접 계산해 넘길 필요가 없고** Seaborn이 알아서 계산합니다.

응용 막대 위의 가는 세로선은 **오차 막대로**, 평균이 얼마나 믿을 만한지를 나타냅니다. 기본값은 95% 신뢰구간입니다.

```
# 평균 대신 중앙값을 보고 싶을 때
sns.barplot(data=df_7, x="라인", y="세정기", estimator="median")
plt.show()
```

```
# 오차 막대를 끄고 싶을 때
sns.barplot(data=df_7, x="라인", y="세정기", errorbar=None)
plt.show()
```

```
▶ errorbar=None을 주면 막대 위 세로선이 사라집니다
```

개념

세 라인의 평균 세정기 온도가 **66.281 · 67.033 · 67.634**로 야간이 가장 낮습니다. 다만 차이가 1.4°C 정도라 **막대 높이만 보면 커 보일 수 있습니다**. 어제 배운 "막대 세로축 자르기" 함정이 여기서도 그대로 적용됩니다.

자주 하는 실수

막대 높이가 평균이라는 사실을 잊기 쉽습니다. 막대가 두 배로 높다고 값이 두 배인 것은 맞지만, **그 그룹의 모든 값이 두 배는 아닙니다**. 그룹 안의 퍼짐을 보려면 barplot이 아니라 boxplot을 써야 합니다.

4-3. palette와 hue — 최신 버전의 규칙

seaborn 0.13.2에서 실제로 나오는 경고

x 예전 방식 — 경고가 뜹니다

```
sns.countplot(data=df, x="라인", palette="Set2")
```

- ▶ FutureWarning: Passing `palette` without assigning `hue` is deprecated and will be removed in v0.14.0.
- ▶ Assign the `x` variable to `hue` and set `legend=False`

바로잡기

✓ 새 방식 — hue에 x와 같은 컬럼, 범례는 끄기

```
sns.countplot(data=df, x="라인", hue="라인",  
              palette="Set2", legend=False)
```

개념

규칙이 바뀐 이유는 **모호함을 없애기 위해서**입니다. 예전에는 palette만 주면 "무엇을 기준으로 색을 나눌지"가 코드에 나타나지 않았습니니다. 이제는 hue로 기준을 명시하게 만든 것입니다.

그래서 색을 쓰는 경우가 **두 가지로 나뉩니다**. 가로축과 같은 기준으로 **색만 바꾸는 경우는** hue=x와 같은 컬럼 + legend=False, **다른 기준으로 색을 나누는 경우는** hue=다른 컬럼이고 범례를 남깁니다.

경우	코드	범례	언제
색만 바꾸기	hue="라인", legend=False	끔	가로축이 이미 라인을 알려줌
기준으로 색 나누기	hue="판정"	유지	색이 어떤 값인지 알려줘야 함

주의

FutureWarning은 **오류가 아니라 예고**입니다. 지금은 동작하지만 **버전 0.14부터 없어진다**는 뜻입니다. 경고를 무시하고 넘어가면 나중에 라이브러리를 올렸을 때 코드가 갑자기 깨집니다. **처음부터 새 방식으로 익히는 편이 낫습니다**.

4-4. 실습 8 — 라인별 불량률

Practice/18_01_example.ipynb — 실습 8

```
df_8 = pd.read_csv("Data/18_열처리.csv", encoding="utf-8")

# groupby와 unstack으로 라인 x 판정 개수 표 만들기
표_개수 = df_8.groupby(["라인", "판정"]).size().unstack(fill_value=0)
print(표_개수)

# 라인마다 총 개수가 다르므로 비율까지 계산
표_개수["합계"] = 표_개수.sum(axis=1)
표_개수["불량률(%)"] = (표_개수["이상"] / 표_개수["합계"] * 100).round(1)
print()
print(표_개수.sort_values("불량률(%)", ascending=False))
```

```
▶ 판정 이상 정상
▶ 라인
▶ 야간 3 96
▶ 주간 7 93
▶ 특근 2 96
▶
▶ 판정 이상 정상 합계 불량률(%)
▶ 라인
▶ 주간 7 93 100 7.0
▶ 야간 3 96 99 3.0
▶ 특근 2 96 98 2.0
```

메서드 `.unstack()` 세로로 긴 표를 가로로 퍼기

정의 `groupby`로 두 기준을 묶으면 결과가 **세로로 길게** 나옵니다. `unstack()`은 안쪽 기준을 **열로 올려** 가로 표로 만듭니다.

형태 `df.groupby(["A", "B"]).size().unstack(fill_value=0)`

```
# unstack 전 — 세로로 길다
print(df_8.groupby(["라인", "판정"]).size())
```

```
▶ 라인 판정
▶ 야간 이상 3
▶ 정상 96
▶ 주간 이상 7
▶ 정상 93
▶ 특근 이상 2
▶ 정상 96
▶ dtype: int64
```

왜 쓰나 세로로 길면 **라인끼리 비교하기 어렵습니다**. 가로로 퍼면 각 행이 라인 하나가 되어 열끼리 나누는 계산(비율)이 가능해집니다.

응용 `fill_value=0`이 중요합니다. 어떤 조합이 아예 없으면 결측이 되는데, **개수 표에서 결측은 0을 의미하기** 때문입니다.

```
# fill_value 없이 하면 없는 조합이 NaN이 됨
t = df_8.groupby(["라인", "판정"]).size().unstack()
print(t.isna().sum().sum(), "개의 빈 칸")
```

```
# pd.crosstab으로 한 줄에 같은 결과
print(pd.crosstab(df_8["라인"], df_8["판정"]))
```

```
▶ 0 개의 빈 칸 (오늘 데이터는 모든 조합이 존재)
▶
▶ 판정 이상 정상
▶ 라인
▶ 야간 3 96
▶ 주간 7 93
▶ 특근 2 96
```

개념

`pd.crosstab()`을 쓰면 `groupby().size().unstack()` 세 단계를 **한 줄로** 끝낼 수 있습니다. 결과가 완전히 같으므로 실무에서는 `crosstab`을 더 자주 씁니다. 다만 **groupby 방식이 더 유연**해서, 개수 말고 평균이나 합계를 넣고 싶을 때는 `groupby`를 씁니다.

설비 적용

주간 불량률 7.0%는 특근 2.0%의 3.5배입니다. 라인별 데이터 개수가 100·99·98로 거의 같아 개수와 비율의 순서가 같지만, 개수가 달랐다면 결론이 뒤집힐 수도 있습니다. 예를 들어 주간 데이터가 500건이었다면 불량 7건은 1.4%로 오히려 가장 낮았을 것입니다.

4-5. 개수가 아니라 비율을 보는 이유

라인	데이터 수	이상 건수	불량률	상자 크기(표준편차)
주간	100	7	7.0%	2.39°C (가장 큼)
야간	99	3	3.0%	1.93°C
특근	98	2	2.0%	1.57°C (가장 작음)

설비 적용

불량률 순서와 표준편차 순서가 정확히 일치합니다. 주간이 가장 흔들리고 가장 많이 불량률이 나며, 특근이 가장 안정적이고 가장 적게 불량률이 납니다. STEP 3의 박스플롯과 STEP 4의 불량률 표가 서로 다른 방법으로 같은 결론에 도달한 것입니다. 이렇게 두 경로가 같은 답을 주면 신뢰도가 크게 올라갑니다.

다만 왜 주간이 더 흔들리는지는 데이터가 답해 주지 않습니다. 작업자 교대, 외기 온도, 설비 예열 상태 등 데이터 밖의 요인이 있을 것입니다. 분석은 "어디를 봐야 하는지"까지만 알려주고, 그다음은 현장의 몫입니다.

STEP 5

scatterplot으로 관계 보기

두 축 · alpha · hue로 위험 영역

여기서 **단변량에서 이변량으로** 넘어갑니다. 온도면 온도, 출력이면 출력을 하나씩 보던 시야를 **둘이 함께 어떻게 움직이는가**로 확장합니다.

5-1. 두 변수의 관계를 보는 이유

목적	무엇	오늘 데이터에서
① 추측	한 센서로 다른 센서를 짐작	온도를 보면 제어출력을 짐작 가능
② 이상 신호	평소 관계가 깨지는 순간	관계가 깨진 지점 = 제어 한계
③ 고장 조건	어떤 조건이 겹칠 때 고장이 나나	불량 12건이 몰린 좌표대

개념

평소 관계가 깨지는 순간이 곧 **이상 신호**라는 것이 이 단원의 핵심입니다. 온도와 출력이 늘 반대로 움직인다면, 어느 날 둘이 같이 오른다면 그것 자체가 신호입니다. 단변량으로는 절대 잡을 수 없는 종류의 신호입니다.

5-2. 실습 1 — 산점도로 관계 확인

함수 `sns.scatterplot()` 두 값을 가로·세로에 두고 점 찍기

정의 x와 y를 모두 지정합니다. `histplot`은 x 하나로 충분했지만 산점도는 두 축이 다 필요합니다.

형태 `sns.scatterplot(data=df, x="숫자1", y="숫자2", hue="범주", alpha=0.5)`

```
df_1 = pd.read_csv("Data/18_열처리.csv", encoding="utf-8")

plt.figure(figsize=(10, 6))
sns.scatterplot(data=df_1, x="소입로온도", y="제어출력", alpha=0.5)
plt.title("소입로온도 - 제어출력 관계")
plt.xlabel("소입로온도(°C)")
plt.ylabel("제어출력")
plt.show()

# 눈으로 본 방향을 숫자로 확인
print("상관계수:", round(df_1["소입로온도"].corr(df_1["제어출력"]), 3))
```

▶ 상관계수: -0.736

▶

▶ 점들이 왼쪽 위에서 오른쪽 아래로 내려가는 띠를 이룹니다

왜 쓰나 점을 잊지 않는 것이 핵심입니다. 297건은 서로 다른 297번의 측정이지 하나의 흐름이 아닙니다. 이으면 없는 순서를 만들어 내는 셈입니다. **관계는 점들이 만드는 전체 모양**에서 읽습니다.

응용 관계가 없는 쌍도 같이 그려 보면 있는 것과의 차이가 분명해집니다.

```
# 비교용 — 관계가 거의 없는 쌍
plt.figure(figsize=(10, 6))
sns.scatterplot(data=df_1, x="소입로온도", y="CP값",
                alpha=0.5, color="gray")
plt.title("소입로온도 - CP값 관계")
plt.show()

print("상관계수:", round(df_1["소입로온도"].corr(df_1["CP값"]), 3))
```

▶ 상관계수: -0.026

▶

▶ 점들이 방향 없이 사각형처럼 퍼져 있습니다

개념

alpha는 **0(완전 투명) ~ 1(완전 불투명)**입니다. 점이 겹치는 곳은 진하게, 흩어진 곳은 연하게 보여 **밀도까지** 드러냅니다. 297개 정도면 alpha=0.5가 적당하고, 수천 개면 0.2 이하로 낮춰야 모양이 보입니다.

주의

축 배치에는 관행이 있습니다. 원인이 될 변수를 x , 결과가 될 변수를 y 에 놓습니다. 축을 바꿔도 관계 자체는 그대로지만, 읽는 사람이 오해하지 않도록 지키는 편이 좋습니다.

5-3. 산점도가 보여주는 세 패턴

점들의 모양	관계	상관계수	오늘 데이터
오른쪽 위로 올라감	양의 관계	+ 부호	—
오른쪽 아래로 내려감	음의 관계	- 부호	소입로온도-제어출력 -0.736
둥글게 흩어짐	관계 없음	0에 가까움	소입로온도-CP값 -0.026

연결

어제 17_열처리.csv에서 온도-OP값 상관이 **-0.783**이었습니다. 오늘 다른 데이터인 18_열처리.csv에서 온도-제어출력이 **-0.736**입니다. 다른 데이터인데 같은 구조가 나온 것입니다. 우연이 아니라 온도 피드백 제어라는 같은 물리 구조가 두 데이터에 다 들어 있기 때문입니다.

제어기가 하는 일이 그대로 보입니다

양 끝을 확인하면 왜 음의 관계인지 알 수 있습니다

```
print(df_1.nlargest(2, "소입로온도")[[ "라인", "소입로온도", "제어출력", "판정"]])
print(df_1.nsmallest(2, "소입로온도")[[ "라인", "소입로온도", "제어출력", "판정"]])
```

- ▶ 라인 소입로온도 제어출력 판정
- ▶ 24 주간 869.3 5.6 이상
- ▶ 191 야간 868.7 0.6 이상
- ▶
- ▶ 라인 소입로온도 제어출력 판정
- ▶ 192 야간 851.4 100.0 이상
- ▶ 209 특근 851.9 100.0 이상

설비 적용

온도가 가장 높은 869.3°C일 때 제어출력 5.6%, 가장 낮은 851.4°C일 때 100%입니다. 제어출력은 히터 출력 비율이므로, 온도가 낮으면 세게 돌리고 높으면 끄는 것입니다. 상관 -0.736은 고장의 증거가 아니라 제어가 정상 작동한다는 증거입니다.

5-4. 실습 2 — hue로 위험 영역 찾기

Practice/18_02_example.ipynb — 실습 2

```
df_2 = pd.read_csv("Data/18_열처리.csv", encoding="utf-8")
```

```
plt.figure(figsize=(10, 6))
sns.scatterplot(data=df_2, x="소입로온도", y="제어출력",
                hue="판정", alpha=0.6)
plt.title("판정별 소입로온도 - 제어출력")
plt.show()
```

판정별 센서 평균 비교

```
print(df_2.groupby("판정")[["소입로온도", "제어출력",
                           "건조출력", "세정기", "CP값"]].mean().round(2))
```

```
▶ 소입로온도  제어출력  건조출력  세정기  CP값
▶ 판정
▶ 이상 856.88   75.14   22.12   67.32   0.45
▶ 정상 859.53   79.17   22.16   66.97   0.45
```

평균만 보면 아무것도 안 보입니다. 이상과 정상의 평균 차이가 온도 2.65°C, 제어출력 4.03, 나머지 셋은 소수점 수준입니다. 그런데 산점도를 보면 **빨간 점들이 명확히 두 군데 구석에 몰려** 있습니다.

오늘 문서에서 가장 중요한 출력

이상 12건이 어디에 있는지 직접 확인

```
ab = df_2[df_2["판정"] == "이상"]
low = ab[ab["소입로온도"] < 855.3]
high = ab[ab["소입로온도"] > 865.4]
```

```
print("저온 이상", len(low), "건 · 제어출력",
      low["제어출력"].min(), "~", low["제어출력"].max())
print("고온 이상", len(high), "건 · 제어출력",
      high["제어출력"].min(), "~", high["제어출력"].max())
```

```
▶ 저온 이상 9 건 · 제어출력 95.7 ~ 100.0
▶ 고온 이상 3 건 · 제어출력 0.6 ~ 5.6
```

설비 적용

이상 12건이 완벽하게 두 덩어리로 나뉩니다. 저온 9건은 제어출력이 전부 95.7% 이상, 고온 3건은 전부 5.6% 이하입니다. 중간은 하나도 없습니다.

이것은 **제어기가 한계까지 밀린 상태**입니다. 히터를 100%로 돌려도 온도가 안 올라가거나, 0%로 꺼도 온도가 안 내려가는 상황입니다. 제어 공학에서 **포화(saturation)**라고 부릅니다.

즉 위험 영역은 "온도가 높은 곳"도 "낮은 곳"도 아니라 **산점도 우하향 띠의 양쪽 끝**입니다. 제어기가 더는 조절할 여지가 없어진 지점입니다.

평균이 아니라 범위와 표준편차를 보십시오

판정별 소입로온도 범위 — 폭이 다릅니다

```
print(df_2.groupby("판정")["소입로온도"].agg(
    ["count", "min", "max", "mean", "std"]).round(2))
```

```
▶      count      min      max      mean      std
▶ 판정
▶ 이상   12   851.4   869.3   856.88   6.91
▶ 정상  285   855.3   865.4   859.53   1.42
```

비교 항목	정상 285건	이상 12건	배수
평균	859.53°C	856.88°C	—
표준편차	1.42°C	6.91°C	4.9배
범위 폭	10.1°C (855.3~865.4)	17.9°C (851.4~869.3)	1.8배

개념

평균은 두 그룹이 비슷하다고 말하고, 표준편차는 완전히 다르다고 말합니다. 이럴 때 믿어야 하는 것은 표준편차입니다. 평균이 비슷한 이유는 이상 12건이 양쪽으로 흩어져 서로 상쇄했기 때문입니다. 한쪽으로만 치우쳤다면 평균에서도 티가 났을 것입니다. **평균 하나로 그룹을 비교하는 것이 왜 위험한지** 보여주는 교과서적인 사례입니다.

STEP 6

상관행렬과 heatmap

corr · annot · fmt · cmap · center

산점도는 두 변수만 봅니다. 변수가 다섯 개면 쌍이 열 개라 산점도를 열 장 그려야 합니다. 이것을 한 장으로 줄이는 것이 heatmap입니다.

6-1. 상관계수 다시 보기

부호와 크기	의미	판단 기준
+ 부호	같이 오르고 같이 내림	양의 관계
- 부호	하나 오르면 하나 내림	음의 관계
절댓값 0.7 이상	강한 관계	거의 함께 움직임
절댓값 0.4 ~ 0.7	중간 관계	경향은 뚜렷함
절댓값 0.2 ~ 0.4	약한 관계	있긴 하지만 흐림
절댓값 0.2 미만	사실상 무관	따로 봐야 함

6-2. 실습 3 — 상관행렬 만들기

메서드 `.corr()` 숫자 컬럼 쌍의 상관계수를 한 번에

정의 모든 숫자 컬럼 쌍의 상관계수를 계산해 **정사각형 표**로 돌려줍니다. 이 표를 상관행렬이라고 합니다.

형태 `df.corr(numeric_only=True)`

```
df_3 = pd.read_csv("Data/18_열처리.csv", encoding="utf-8")
```

```
corr_3 = df_3.corr(numeric_only=True)
print(corr_3.round(3))
```

```
▶ 소입로온도  제어출력  건조출력  세정기  CP값
▶ 소입로온도  1.000  -0.736  -0.007   0.032  -0.026
▶ 제어출력   -0.736   1.000   0.088  -0.019   0.009
▶ 건조출력   -0.007   0.088   1.000  -0.053  -0.095
▶ 세정기     0.032  -0.019  -0.053   1.000  -0.043
▶ CP값      -0.026   0.009  -0.095  -0.043   1.000
```

왜 쓰나 변수 5개면 쌍이 10개입니다. 하나씩 계산하면 열 줄을 써야 하지만 `corr()` 한 줄이면 끝납니다. 21일차에 배운 그 메서드가 여기서 heatmap의 재료가 됩니다.

응용 `numeric_only=True`가 없으면 글자형 컬럼 때문에 오류가 나거나 경고가 뜹니다. 그리고 대각선과 대칭을 알면 절반만 봐도 됩니다.

```
# 대각선을 빼고 절댓값이 큰 순으로 정렬
쌍 = corr_3.unstack()
쌍 = 쌍[쌍.index.get_level_values(0) < 쌍.index.get_level_values(1)]
print(쌍.reindex(쌍.abs().sort_values(ascending=False).index).round(3))
```

```
▶ 소입로온도 제어출력 -0.736
▶ CP값 건조출력 -0.095
▶ 건조출력 제어출력 0.088
▶ 건조출력 세정기 -0.053
▶ CP값 세정기 -0.043
▶ 세정기 소입로온도 0.032
▶ CP값 소입로온도 -0.026
▶ 세정기 제어출력 -0.019
▶ CP값 제어출력 0.009
▶ 건조출력 소입로온도 -0.007
```

개념

상관행렬에는 두 가지 규칙이 있습니다. ① **대각선은 항상 1.000**입니다. 자기 자신과는 완벽하게 같이 움직이기 때문입니다. ② **대각선 기준 위아래가 대칭**입니다. 온도-출력과 출력-온도가 같은 값이기 때문입니다. 그래서 **절반만 봐도 모든 정보를 얻습니다**.

설비 적용

10쌍 중 뚜렷한 것은 소입로온도-제어출력 -0.736 하나뿐입니다. 나머지 9쌍은 전부 절댓값 0.1 미만으로 사실상 무관합니다. 센서가 5개라도 **실제로 연동되는 쌍은 1개**라는 뜻이고, 이는 나머지 센서들이 **서로 독립적으로 제어**된다는 의미입니다. 각각을 따로 감시해야 합니다.

6-3. heatmap으로 색칠하기

함수 `sns.heatmap()` 상관행렬을 색으로 칠하기

정의 표의 각 칸을 **값에 따라 색칠**합니다. 입력이 원본 데이터프레임이 아니라 **상관행렬**이라는 점이 가장 헷갈리는 부분입니다.

형태 `sns.heatmap(corr, annot=True, fmt=".2f", cmap="coolwarm", center=0)`

```
corr_3 = df_3.corr(numeric_only=True)

plt.figure(figsize=(8, 6))
sns.heatmap(data=corr_3, annot=True, fmt=".2f",
            cmap="coolwarm", center=0)
plt.title("센서 상관관계 heatmap")
plt.show()
```

- ▶ 5×5 격자에 -0.74부터 1.00까지의 값이 색과 숫자로 표시됩니다
- ▶ 대각선은 전부 1.00으로 가장 진한 빨강입니다

왜 쓰나 사람은 숫자 25개를 비교하는 데 약하고 색 25개를 비교하는 데 강합니다. 변수가 10개가 되면 칸이 100개인데, 숫자로써는 어디가 강한지 절대 한눈에 안 보입니다.

응용 네 개의 옵션이 **한 세트**로 함께 쓰입니다. 하나라도 빠지면 읽기 어려워집니다.

```
# annot 없이 — 색만 보이고 정확한 값을 모름
sns.heatmap(corr_3)

# fmt 없이 annot만 — 0.51234처럼 길게 나와 지저분
sns.heatmap(corr_3, annot=True)

# center 없이 — 약한 음수도 진하게 보여 오해를 부름
sns.heatmap(corr_3, annot=True, fmt=".2f", cmap="coolwarm")

# 완성형 — 네 개를 모두
sns.heatmap(corr_3, annot=True, fmt=".2f",
            cmap="coolwarm", center=0)
```

개념

`center=0`이 특히 중요합니다. **0을 색의 한가운데로** 고정해 양수는 빨강, 음수는 파랑, 0에 가까우면 흰색으로 나오게 합니다. 이 옵션이 빠지면 데이터의 최솟값이 가장 진한 파랑이 되어, **-0.09 같은 약한 음수도 진하게 보여** 관계가 있는 것처럼 오해하게 됩니다.

자주 하는 실수

`sns.heatmap(df)`처럼 **원본 데이터프레임을 그대로 넣는 실수**가 가장 흔합니다. 그러면 297행 5열이 그대로 색칠되어 아무 의미 없는 세로 띠가 나옵니다. **반드시** `corr()` 결과를 넣으십시오.

6-4. heatmap 옵션 정리

옵션	값	하는 일	빠면
annot=	True	각 칸에 숫자 표시	색만 보이고 값을 모름
fmt=	".2f"	소수점 두 자리로	0.51234처럼 길게 나옴
cmap=	"coolwarm"	빨강-파랑 색 지도	기본 색이라 방향이 안 보임
center=	0	0을 색의 중앙으로	약한 음수도 진하게 보임
figsize=	(8, 6)	그림 크기	칸이 작아 숫자가 겹침

주의

heatmap의 행·열 이름이 **한글**이므로 폰트 설정이 안 되어 있으면 축 전체가 네모로 깨집니다. 그리고 상관계수에는 **음수**가 있으므로 `axes.unicode_minus = False`도 반드시 필요합니다. STEP 1의 준비 두 줄이 여기서 결정적으로 쓰입니다.

6-5. 상관은 원인이 아닙니다

☒ 단정 표현	☒ 사실 표현
"온도가 제어출력의 원인이다"	"온도와 제어출력은 반대로 움직이는 경향"
"이 설비는 고장이다"	"이 설비의 온도가 정상 범위 밖"
"저온이 반드시 불량을 만든다"	"정상 때 밖 영역에 불량이 몰림"

개념

상관계수 -0.736 은 **온도와 출력이 함께 움직인다**는 사실만 말합니다. 온도가 출력을 낮춘 것도, 출력이 온도를 올린 것도 아닙니다. 둘 사이에 **제어기**라는 장치가 있어 서로를 조절하는 것입니다. 데이터만 봐서는 이 구조를 알 수 없고 **공정을 알아야** 읽힙니다. 그래서 리포트에는 **그래프가 실제로 보여준 것만** 적어야 합니다.

STEP 7

시각화 리포트 만들기

subplots · ax · savefig · 사실→의미→행동

그래프는 수단이고 목적은 **발견과 전달**입니다. 여러 그래프를 모아 "이 데이터에서 무엇을 발견했다"를 말하는 것이 시각화 리포트입니다.

7-1. 리포트 구성 순서



단계	묻는 질문	쓰는 그래프	오늘 데이터의 답
①	데이터가 어떻게 생겼나	countplot	297건 · 라인 100/99/98
②	각 센서의 정상 범위는	histplot boxplot	온도 858.8~860.2 (IQR)
③	그룹 간 차이가 있나	boxplot(hue)	주간 불량을 7.0%로 최고
④	센서끼리 연동되나	scatterplot heatmap	온도-출력 -0.736 하나뿐

개념

큰 그림에서 점점 좁혀 가는 것이 자연스러운 순서입니다. 영화가 배경을 먼저 보여주고 인물 관계를 나중에 풀듯, 리포트도 전체 규모부터 시작해 개별 분포, 그룹 비교, 변수 관계 순으로 갑니다. 순서가 뒤집히면 **보는 사람이 길을 잃습니다**.

7-2. 실습 4 — subplots로 가로 3칸

함수 `plt.subplots()` 격자를 만들고 축 객체를 받기

정의 격자를 한 번에 만들고 **전체 도화지(fig)**와 **각 칸(axes)**을 함께 돌려줍니다. 어제 배운 `plt.subplot()`(단수)과 이름이 한 글자 차이입니다.

형태 `fig, axes = plt.subplots(행, 열, figsize=(가로, 세로))`

```

df_4 = pd.read_csv("Data/18_열처리.csv", encoding="utf-8")

# 가로 3칸 만들기
fig, axs = plt.subplots(1, 3, figsize=(16, 4))

# seaborn 함수는 ax= 로 어느 칸에 그릴지 지정
sns.countplot(data=df_4, x="라인", hue="라인",
               palette="Set2", legend=False, ax=axs[0])
axs[0].set_title("라인별 개수")

sns.histplot(data=df_4, x="소입로온도", bins=20, kde=True, ax=axs[1])
axs[1].set_title("소입로온도 분포")

sns.boxplot(data=df_4, x="라인", y="건조출력",
             hue="라인", palette="pastel", legend=False, ax=axs[2])
axs[2].set_title("라인별 건조출력 박스플롯")

plt.tight_layout()
plt.show()

```

▶ 가로로 나란한 세 칸에 개수·분포·박스플롯이 배치됩니다

왜 쓰나 그림 세 장을 따로 붙이는 것보다 한 장에 모으면 서로 연결해서 읽힙니다. 그리고 보고서에 넣을 때도 파일 하나면 됩니다.

응용 Seaborn 함수에서는 어느 칸에 그릴지를 `ax=`로 지정합니다. 어제 `plt.subplot()`처럼 "현재 칸"을 바꾸는 방식이 아니라 **칸을 직접 가리키는** 방식입니다.

```

# 1행이면 axs는 1차원 — axs[0], axs[1], axs[2]
fig, axs = plt.subplots(1, 3)

# 2행 2열이면 axs는 2차원 — axs[0][0], axs[0][1] ...
fig, axs = plt.subplots(2, 2)

# 칸이 하나면 axs는 배열이 아니라 축 객체 하나
fig, ax = plt.subplots(1, 1)

```

개념

축 객체에서는 함수 이름이 달라집니다. `plt.title()`이 `axs[0].set_title()`이 되고, `plt.xlabel()`이 `axs[0].set_xlabel()`이 됩니다. 앞에 `set_`이 붙는다고 기억하면 됩니다. 이것이 축 객체 방식에서 가장 자주 걸리는 부분입니다.

주의

`plt.tight_layout()`을 빼면 칸끼리 붙어 제목과 축 이름이 겹칩니다. 여러 칸을 쓸 때는 거의 항상 필요하다고 생각하십시오. 그리고 모든 그래프를 그린 뒤에, `show`나 `savefig` 앞에서 호출해야 합니다.

7-3. plt 방식과 ax 방식 비교

하는 일		
칸 만들기	<code>plt.subplot(2,2,1)</code>	<code>fig, axs = plt.subplots(2,2)</code>
칸 번호	1부터 시작	0부터 시작
그리기	<code>plt.hist(y)</code>	<code>sns.histplot(..., ax=axs[0][0])</code>
제목	<code>plt.title("...")</code>	<code>axs[0][0].set_title("...")</code>
축 이름	<code>plt.xlabel("...")</code>	<code>axs[0][0].set_xlabel("...")</code>
전체 제목	—	<code>fig.suptitle("...")</code>

자주 하는 실수

두 방식을 섞으면 헷갈립니다. `plt.subplot()`의 번호는 1부터, `axs`의 인덱스는 0부터입니다. 그리고 `plt.title()`은 현재 활성화된 칸에 제목을 달기 때문에, `ax=`로 그린 그래프에 `plt.title()`을 쓰면 엉뚱한 칸에 제목이 붙습니다. Seaborn을 쓸 때는 **ax** 방식으로 통일하는 편이 안전합니다.

7-4. 실습 5 — 관계 그래프 두 칸

Practice/18_02_example.ipynb — 실습 5

```
df_5 = pd.read_csv("Data/18_열처리.csv", encoding="utf-8")

fig, axs = plt.subplots(1, 2, figsize=(14, 5))

# 왼쪽 — 판정별 산점도
sns.scatterplot(data=df_5, x="소입로온도", y="제어출력",
                hue="판정", alpha=0.6, ax=axs[0])
axs[0].set_title("판정별 소입로온도 - 제어출력")
axs[0].set_xlabel("소입로온도(°C)")
axs[0].set_ylabel("제어출력")

# 오른쪽 — 상관 heatmap
corr_5 = df_5.corr(numeric_only=True)
sns.heatmap(data=corr_5, annot=True, fmt=".2f",
            cmap="coolwarm", center=0, ax=axs[1])
axs[1].set_title("센서 상관관계")

plt.tight_layout()
plt.show()
```

▶ 왼쪽 산점도의 우하향 띠와 오른쪽 heatmap의 -0.74 칸이 나란히 놓입니다

개념

같은 사실을 두 방식으로 나란히 보여주는 구성입니다. 왼쪽 산점도는 "우하향 띠"라는 모양으로, 오른쪽 heatmap은 "-0.74"라는 숫자로 같은 말을 합니다. 그림과 숫자를 나란히 두면 **서로를 검증**해 줍니다. 산점도가 우하향인데 상관관계수가 양수라면, 둘 중 하나는 코드가 잘못된 것입니다.

7-5. 실습 6 — 2×2 리포트와 저장

Practice/18_02_example.ipynb — 실습 6

```
df_6 = pd.read_csv("Data/18_열처리.csv", encoding="utf-8")

fig, axs = plt.subplots(2, 2, figsize=(14, 10))
fig.suptitle("18_02 열처리 공정 시각화 리포트", fontsize=16)

# ① 분포
```

```

sns.histplot(data=df_6, x="소입로온도", hue="판정",
              bins=20, kde=True, ax=axes[0][0])
axes[0][0].set_title("판정별 소입로온도 분포")

# ② 박스플롯
sns.boxplot(data=df_6, x="라인", y="소입로온도",
             hue="판정", ax=axes[0][1])
axes[0][1].set_title("라인 x 판정 소입로온도 박스플롯")

# ③ 산점도
sns.scatterplot(data=df_6, x="소입로온도", y="제어출력",
                 hue="판정", alpha=0.6, ax=axes[1][0])
axes[1][0].set_title("판정별 소입로온도 - 제어출력")

# ④ heatmap
sns.heatmap(data=df_6.corr(numeric_only=True), annot=True,
             fmt=".2f", cmap="coolwarm", center=0, ax=axes[1][1])
axes[1][1].set_title("센서 상관관계")

plt.tight_layout()

# savefig가 show보다 먼저!
plt.savefig("18_02_실습6_시각화리포트.png", dpi=150, bbox_inches="tight")
plt.show()

```

▶ 한 장에 분포·박스·관계·상관이 담긴 2x2 리포트가 파일로 저장됩니다

함수 plt.savefig() 그래프를 이미지 파일로 저장

정의 현재 그림을 파일로 씁니다. **반드시** plt.show()보다 먼저 **호출해야** 합니다.

형태 plt.savefig("파일명.png", dpi=150, bbox_inches="tight")

```

# x 잘못된 순서 — 빈 파일이 저장됩니다
plt.show()
plt.savefig("리포트.png")

# ✓ 올바른 순서
plt.savefig("리포트.png", dpi=150, bbox_inches="tight")
plt.show()

```

▶ 잘못된 순서로 저장한 파일을 열면 하얀 빈 이미지가 나옵니다

왜 쓰나 plt.show()가 그림을 화면에 띄운 뒤 비워 버리기 때문입니다. 비워진 다음에 저장하면 남은 것이 없습니다.

이 순서 실수는 **오류가 나지 않아** 파일을 열어 보기 전까지 모릅니다.

응용 dpi는 선명도, bbox_inches="tight"는 여백 잘라내기입니다. 확장자에 따라 형식이 자동으로 정해집니다.

```
# 리포트용 — 글자가 또렷한 png
plt.savefig("리포트.png", dpi=150, bbox_inches="tight")

# 인쇄용 — 확대해도 안 깨지는 pdf
plt.savefig("리포트.pdf", bbox_inches="tight")

# 용량 절약 — jpg
plt.savefig("리포트.jpg", dpi=100, bbox_inches="tight")
```

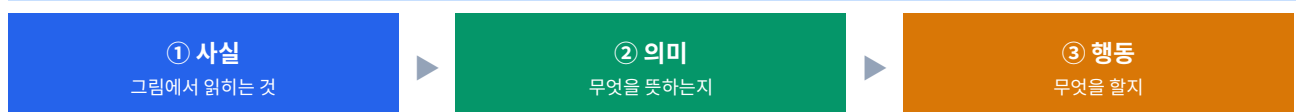
개념

dpi는 dots per inch로 **1인치에 점을 몇 개 찍을지**입니다. 화면용은 100, 리포트용은 150, 인쇄용은 300을 씁니다. 숫자가 클수록 선명하지만 파일도 커집니다.

주의

bbox_inches="tight"를 빼면 그림 주위에 **하얀 여백이 넓게** 남습니다. 문서에 붙일 때 여백 때문에 배치가 어긋나므로 거의 항상 넣는 것이 좋습니다.

7-6. 발견을 문장으로 — 사실 → 의미 → 행동



그래프만 있는 리포트는 반쪽입니다. 각 그래프에서 알아낸 것을 글로 적어야 비로소 리포트가 됩니다. 오늘 데이터로 세 단계를 실제로 써 보면 이렇습니다.

그래프	① 사실	② 의미	③ 행동
히스토그램	이상 12건이 정상 온도 띠 밖에 100% 존재	정상 범위 안에서는 불량인 양	855.3~865.4°C를 관리 기준으로
박스플롯	주간 표준편차 2.39로 가장 큼	주간 온도 제어가 가장 흔들림	주간 근무 조건 점검
산점도	이상 12건 모두 제어출력 극단값	제어기가 한계까지 밀림	출력 95% 이상 지속 시 알람

그래프	① 사실	② 의미	③ 행동
heatmap	10쌍 중 뚜렷한 것은 1쌍뿐	나머지 센서는 서로 독립	센서별로 따로 감시

주의

③ 행동은 제안이지 결론이 아닙니다. "출력 95% 이상 지속 시 알람"은 데이터가 보여준 패턴에서 나온 제안일 뿐, 실제로 유효한 기준인지는 현장이 판단합니다. 리포트를 쓸 때 ①은 단정해도 되고, ②는 조심스럽게, ③은 제안으로 적는 것이 원칙입니다.

7-7. 보는 사람에 따라 그래프를 고르기

대상	관심사	넣을 그래프	오늘 데이터에서
설비 담당자	어디를 점검해야 하나	박스플롯 · 산점도	주간 라인 · 제어출력 극단 구간
경영진	전체 상황이 어떤가	barplot · countplot	불량률 주간 7.0% · 전체 4.0%

개념

같은 데이터라도 누가 보느냐에 따라 중요한 그래프가 다릅니다. 설비 담당자에게 heatmap을 보여주면 "그래서 뭘 고치라는 건가"가 되고, 경영진에게 산점도를 보여주면 "점이 많다"로 끝납니다. EDA에서 그린 수십 장을 다 넣지 말고, 발견을 담은 것만 골라 넣으십시오. 좋은 리포트는 그래프 수가 아니라 메시지의 명확함으로 결정됩니다.

여기까지가 Seaborn 부분입니다. 한 줄로 그릴 수 있게 되니 여덟 장을 그려 보게 되었고, 그 결과 평균만 봐서는 절대 못 찾았을 사실을 찾았습니다. 이상 12건이 정상 때 밖에 100% 존재하고, 그때 제어가 포화 상태였다는 것 — 이것이 오늘 앞부분의 성과입니다.

STEP 8

시계열 데이터란

순서의 의미 · timestamp와 cycle

여기서 데이터의 성격이 완전히 바뀝니다. 지금까지는 행 순서가 아무 의미 없었지만, 이제부터는 순서가 정보 그 자체입니다.

8-1. 순서를 섞으면 사라지는 정보

구분		
예시	학생 키 목록	1분마다 기록한 설비 온도
데이터	김철수 172 · 이영희 165 · 박민수 180	09:00 → 80.1 · 09:01 → 80.3 · 09:02 → 80.6
섞으면	키 정보는 그대로 유지	"점점 올라가고 있었다"가 사라짐
핵심 질문	값이 얼마인가	어떻게 변화하는가
분석 초점	평균 · 분포	추세 · 변동
오늘 데이터	18_열처리.csv	20_cmapss_fd001_sample.csv

개념

"이 데이터는 행의 순서가 의미를 가지는가." 데이터를 받았을 때 가장 먼저 던져야 할 질문입니다. 그렇다면, 그리고 그 순서가 시간이라면, 시계열 분석으로 접근해야 합니다. 설비 데이터의 순서는 거의 항상 시간 축입니다.

8-2. 시계열 데이터의 정의

시계열은 시간 순서대로 기록되고, 모든 값에 시간 정보가 함께 붙은 데이터입니다. 두 가지 조건이 다 필요합니다.

조건	무엇	빠지면
① 시간 축	"언제 측정됐는가"가 붙어 있음	값만 있으면 시계열이 아님
② 짝지음	하나의 시점에 하나의 값	어느 값이 언제인지 모름

날씨·주가·미세먼지·심전도 — 모두 시간 축 위에 값이 늘어선 시계열입니다. 그리고 설비 데이터는 본질적으로 시계열입니다. 설비가 쉬지 않고 돌면서 센서가 정해진 간격마다 값을 남기기 때문입니다.

8-3. 흐름으로 봐야 보이는 신호

보는 방식	판단	놓치는 것
한 시점만 보기	"이 정도 진동은 정상인데?"	값은 정상 범위인데 계속 오르는 중
흐름으로 보기	"정상 범위지만 계속 올라간다"	—

설비 적용

고장은 한순간에 일어나는 듯 보여도 데이터에는 흔적이 남습니다. 그리고 그 흔적은 시간 순서로 봐야만 보입니다. 한 시점의 값이 정상 범위 안에 있어도, 그 값이 열 번의 작동에 걸쳐 계속 올라가고 있다면 이미 신호입니다. 이것이 설비 데이터를 시계열로 다루는 본질적 이유입니다.

8-4. 시계열 분석의 세 가지 목적지



목적지	무엇	설비에서의 의미	신호 강도
추세	시간에 따라 올라가는가 내려가는가	부품 마모 가능성	가장 먼저 잡히는 약한 신호
변동성	값이 얼마나 출렁이는가	고장이 가까울수록 커짐	중간 단계의 신호
이상 징후	평소 패턴을 벗어난 급격한 틱	즉시 대응 필요	가장 강한 신호

개념

세 신호는 보통 이 순서로 강해집니다. 추세가 가장 먼저 나타나고, 그다음 변동성이 커지고, 마지막에 이상치가 틱니다. 그래서 추세를 빨리 감지할수록 정비 시간을 확보할 수 있습니다. 이상치가 튼 뒤에 발견하면 이미 늦은 경우가 많습니다.

8-5. 네 가지 패턴의 모양

패턴	그래프 모양	비유	뜻
정상	잔잔한 가로 흐름	잔잔한 호수 수면	일정 수준 근처에서 안정
추세 이상	우상향 또는 우하향	한 방향으로 밀려남	값이 시간 따라 이동
변동성 이상	출렁임이 점점 커짐	파도가 거세짐	범위가 넓어짐

패턴	그래프 모양	비유	뜻
이상치	한 점만 뽀족하게 튀	스파이크	한 점만 갑자기 다름

주의

모든 변화가 곧바로 고장을 뜻하지는 않습니다. 설비를 세계 가동하거나 날씨가 더우면 값은 자연히 변합니다. 판단 기준은 "같은 작동 조건인데도 흐름이 달라졌는가"입니다. 그래서 실제 설비에서는 세 가지 패턴이 섞여 나타나고, 정상 패턴을 데이터로 충분히 익혀야 비정상이 보입니다.

8-6. 두 가지 시간 축 — timestamp와 cycle

구분		
시간 축	실제 날짜와 시각	몇 번째 작동인가
예시	2024-03-01 09:00	137번째
대표 데이터	공장 · 발전소 · 건물 에너지	NASA C-MAPSS
가능한 분석	시 · 일 · 월 단위 집계	이동평균 · 변화량
답하는 질문	"몇 시에 온도가 가장 높은가"	"몇 번 더 쓸 수 있는가"
현장 용도	실시간 감시 · 운영 분석	수명 예측 · 설비 비교

비유

cycle은 자동차 주행거리와 같은 발상입니다. 차가 얼마나 닳았는지는 "몇 년 됐나"보다 "몇 km 탔나"가 더 정확합니다. 한 달에 한 번 타는 차와 하루에 백 번 타는 차는 같은 한 달이라도 닳음 정도가 전혀 다릅니다. 설비도 마찬가지로 작동 횟수를 시간처럼 다루는 것입니다.

개념

두 방식의 공통점은 둘 다 순서가 살아 있고, 섞으면 정보가 사라지며, 추세 · 변동 · 이상 탐지라는 목적이 같다는 것입니다. 차이점은 시간 축이 실제 시각이나 작동 횟수냐이고, 그에 따라 쓸 수 있는 도구가 달라집니다. cycle 데이터로는 시 · 일 · 월 집계를 할 수 없습니다. "1 cycle = 1시간"으로 약속하면 변환은 가능합니다.

8-7. 예지보전이 목표입니다



방식	언제 고치나	문제점
사후 정비	고장 난 다음에	갑작스러운 생산 중단
예방 정비	정해진 일정대로	덜컾한 부품도 교체해 낭비
예지보전	신호가 나타났을 때	—

설비 적용

이 과정에서 배우는 모든 것이 결국 예지보전을 위한 기초 체력입니다. 실시간 감시는 timestamp로, 수명 가늠은 cycle로 합니다. 다만 이번 단원에서 모델로 고장을 예측하는 데까지 가지는 않습니다. 우선은 데이터에서 추세·변동성·이상 징후를 직접 눈으로 찾아내는 능력을 만드는 것이 목표입니다.

STEP 9

C-MAPSS 구조 읽기

엔진 5대 · 센서 21개 · 실습 1~4

NASA가 시뮬레이션으로 만든 **예지보전 표준 데이터셋**입니다. 엔진이 고장 날 때까지의 전 과정이 통째로 담겨 있어 추세와 변동성 공부에 최적입니다.

9-1. 실습 1 — 데이터 열고 구조 확인

Practice/20_01_example.ipynb — 실습 1

```
df_1 = pd.read_csv("Data/20_cmapss_fd001_sample.csv")
```

```
print("shape:", df_1.shape)
print("columns:", list(df_1.columns))
print()
print(df_1.head())
```

```
▶ shape: (1116, 26)
▶ columns: ['unit_nr', 'time_cycles', 'setting_1', 'setting_2',
            'setting_3', 's_1', 's_2', ..., 's_21']
```

류음	열 이름	개수	의미
엔진 번호	unit_nr	1	어느 엔진의 기록인가
작동 cycle	time_cycles	1	몇 번째 작동인가 — 시간 축
운전 설정	setting_1 ~ 3	3	작동 조건
센서값	s_1 ~ s_21	21	온도 · 압력 · 회전수 등

개념

센서 이름이 s_1, s_2처럼 번호뿐입니다. 무엇을 재는 센서인지 알려주지 않습니다. 처음에는 불친절해 보이지만, 패턴만으로 좋은 센서를 판단하는 훈련이 된다는 점에서 오히려 좋은 교재입니다. 실제 현장에서도 센서 이름이 없거나 의미를 모르는 경우가 흔합니다.

9-2. 실습 2 — 여러 엔진이 섞인 구조

메서드 `.nunique()` 중복을 뺀 고유값의 개수

정의 컬럼 안에서 중복을 제외한 값이 몇 종류인지 셉니다. `value_counts()`가 각 값의 개수를 세는 것과 다릅니다.

형태 `df["컬럼"].nunique()`

```
df_2 = pd.read_csv("Data/20_cmapss_fd001_sample.csv")

print("엔진 대수:", df_2["unit_nr"].nunique())
print()
print("[엔진별 작동 회차 수]")
길이_2 = df_2["unit_nr"].value_counts().sort_index()
print(길이_2)

print()
print("가장 짧은 엔진:", 길이_2.idxmin(), "번 /", 길이_2.min(), "회차")
print("가장 긴 엔진 :", 길이_2.idxmax(), "번 /", 길이_2.max(), "회차")
print("전체 합계   :", 길이_2.sum(), "행")
```

```
▶ 엔진 대수: 5
▶
▶ [엔진별 작동 회차 수]
▶ unit_nr
▶ 1      192
▶ 2      287
▶ 3      179
▶ 4      189
▶ 5      269
▶
▶ 가장 짧은 엔진: 3 번 / 179 회차
▶ 가장 긴 엔진  : 2 번 / 287 회차
▶ 전체 합계     : 1116 행
```

왜 쓰나 1116행이 엔진 하나의 기록이 아니라 다섯 대가 섞인 것임을 확인하는 단계입니다. 이걸 모르고 전체를 한 그래프에 그리면 다섯 엔진의 흐름이 뒤엉켜 아무것도 안 보입니다.

응용 `.idxmin()` · `.idxmax()`는 **최솟값**·**최댓값**이 있는 **인덱스**를 돌려줍니다. 값이 아니라 이름을 얻는다는 점이 `.min()`·`.max()`와 다릅니다.

```
# 값과 인덱스의 차이
print("최솟값:", 길이_2.min())      # 179
print("그 인덱스:", 길이_2.idxmin()) # 3

# 전체 합계가 df 행 수와 같은지 검사
print(길이_2.sum() == len(df_2))

▶ 최솟값: 179
▶ 그 인덱스: 3
▶ True
```

개념

엔진마다 작동 회차 수가 다릅니다. 179회에서 287회까지 차이가 납니다. C-MAPSS는 각 엔진을 **고장 날 때까지** 돌린 데이터이므로, 회차 수가 곧 **그 엔진의 수명**입니다. 3번 엔진이 179회에 고장 났고 2번 엔진은 287회까지 버틴 것입니다.

설비 적용

같은 모델의 엔진인데 수명이 **1.6배** 차이 납니다. 이것이 **개체차**입니다. 그래서 예지보전은 "이 모델은 200회에 고장 난다"가 아니라 "**이 엔진은 지금 어떤 상태인가**"를 봐야 합니다. 모델 평균이 아니라 개별 설비의 신호를 봐야 하는 이유입니다.

9-3. 실습 3 — 요약 통계로 센서 범위 읽기

Practice/20_01_example.ipynb — 실습 3

```
df_3 = pd.read_csv("Data/20_cmapss_fd001_sample.csv")

# 2번 엔진을 골라 세 센서의 요약 통계
engine_3 = df_3[df_3["unit_nr"] == 2]
print("2번 엔진 행 수:", len(engine_3))
print()
print(engine_3[["s_2", "s_11", "s_12"]].describe().round(2))

print()
for sensor in ["s_2", "s_11", "s_12"]:
    s = engine_3[sensor]
    print(f"{sensor:5} 평균 {s.mean():8.2f} | 표준편차 {s.std():5.2f}"
          f" | 범위 {s.min():.2f} ~ {s.max():.2f} (폭 {s.max()-s.min():.2f})")
```

▶ 2번 엔진 행 수: 287

```

>
>          s_2      s_11      s_12
> count    287.00    287.00    287.00
> mean     641.75     47.69    517.43
> std       0.49      0.32      2.15
> min      640.46     46.97    512.86
> max      642.87     48.44    521.96
>
> s_2   평균 641.75 | 표준편차 0.49 | 범위 640.46 ~ 642.87 (폭 2.41)
> s_11  평균 47.69 | 표준편차 0.32 | 범위 46.97 ~ 48.44 (폭 1.47)
> s_12  평균 517.43 | 표준편차 2.15 | 범위 512.86 ~ 521.96 (폭 9.10)

```

개념

평균이 641.75 · 47.69 · 517.43으로 크기가 제각각입니다. 같은 축에 그리면 s_11(약 48)이 s_2(약 642) 아래에 납작하게 깔려 보이지도 않습니다.

그리고 표준편차도 0.49 · 0.32 · 2.15로 다릅니다. 그런데 **표준편차만 비교하면 공평하지 않습니다.** s_12의 2.15는 평균 517 대비 0.4%지만, s_11의 0.32는 평균 47.7 대비 0.7%입니다. **평균 대비 비율로 봐야** 어느 센서가 더 흔들리는지 알 수 있습니다.

스케일을 맞춰 비교하기

```

# 평균 대비 흔들림 비율 — 변동계수
for sensor in ["s_2", "s_11", "s_12"]:
    s = engine_3[sensor]
    print(f"{sensor:5} 표준편차/평균 = {s.std()/s.mean()*100:.3f}%")

```

```

> s_2   표준편차/평균 = 0.076%
> s_11  표준편차/평균 = 0.662%
> s_12  표준편차/평균 = 0.415%

```

절대 표준편차로는 s_12가 가장 크지만, 비율로는 s_11이 가장 큼니다. 순서가 뒤집힌 것입니다. 크기가 다른 센서를 비교할 때는 반드시 이 점을 확인해야 합니다. **9000짜리 센서의 10 변화와 8짜리 센서의 10 변화는 전혀 다른 의미입니다.**

9-4. 실습 4 — 쓸 센서 고르기

Practice/20_01_example.ipynb — 실습 4

```

df_4 = pd.read_csv("Data/20_cmapss_fd001_sample.csv")

sensors = ["s_1", "s_6", "s_7", "s_11", "s_12"]
for sensor in sensors:
    print(sensor, round(df_4[sensor].std(), 3))

```

```
# 전체를 훑어 자동으로 골라내기
std_4 = df_4.std(numeric_only=True).round(4)

제외_4 = std_4[std_4 == 0].index.tolist()
사용_4 = [c for c in std_4.index
           if c not in 제외_4 and c not in ["unit_nr", "time_cycles"]]

print("제외 대상:", 제외_4)
print(f"-> 실제로 쓸 수 있는 건 {len(사용_4)}개")

print("결측 합계:", df_4.isna().sum().sum())
```

```
▶ s_1 0.0
▶ s_6 0.0
▶ s_7 2.367
▶ s_11 0.314
▶ s_12 2.119
▶
▶ 제외 대상: ['setting_3', 's_1', 's_5', 's_6', 's_10', 's_16', 's_18', 's_19']
▶ -> 실제로 쓸 수 있는 건 16개
▶ 결측 합계: 0
```

구분	개수	목록 또는 설명
전체 센서	21개	s_1 ~ s_21
상수 센서	7개	s_1, s_5, s_6, s_10, s_16, s_18, s_19
쓸 수 있는 센서	14개	값이 실제로 변하는 센서
운전 설정	2개	setting_3도 상수라 제외
분석 대상 합계	16개	센서 14 + 설정 2

설비 적용

21개 센서 중 7개가 처음부터 끝까지 같은 값입니다. s_1은 항상 518.67, s_5는 항상 14.62입니다. 이런 컬럼은 그래프를 그려도 직선만 나오고 상관계수는 NaN이 됩니다. 정보가 0이므로 과감히 빼야 합니다. 실제 현장에서도 고장 난 센서나 사용하지 않는 채널이 상수로 남아 있는 경우가 많습니다.

9-5. 부동소수점 함정 — 실습 8 코드가 알려주는 것

같은 상수인데 결과가 다릅니다

```
# round 없이 std == 0 으로 비교하면?
std_raw = df_4.std(numeric_only=True)
print("round 없이:", std_raw[std_raw == 0].index.tolist())

# 실제 값을 찍어 보면
for c in ["s_1", "s_5", "s_6", "s_10", "s_16", "s_18", "s_19"]:
    print(f"{c:5} std = {df_4[c].std():.6g}   값 종류 {df_4[c].nunique()}개")
```

```
▶ round 없이: ['setting_3', 's_1', 's_18', 's_19']
```

```
▶
```

```
▶ s_1    std = 0                값 종류 1개
```

```
▶ s_5    std = 1.77715e-15      값 종류 1개
```

```
▶ s_6    std = 7.10861e-15      값 종류 1개
```

```
▶ s_10   std = 2.22144e-16      값 종류 1개
```

```
▶ s_16   std = 3.471e-18        값 종류 1개
```

```
▶ s_18   std = 0                값 종류 1개
```

```
▶ s_19   std = 0                값 종류 1개
```

자주 하는 실수

여덟 개 모두 값 종류가 1개인 상수인데, `std == 0`으로 걸러 보면 네 개만 잡힙니다. 나머지 넷은 표준편차가 정확히 0이 아니라 $1.78\text{e-}15$ 같은 극히 작은 값이기 때문입니다.

컴퓨터가 실수를 2진수로 근사해서 저장하기 때문에 생기는 오차입니다. $0.1 + 0.2 == 0.3$ 이 False가 되는 것과 같은 원리입니다. 값이 다 같아도 뺄셈과 제곱근을 거치면 미세한 찌꺼기가 남습니다.

그래서 `.round(4)`로 자릿수를 맞춘 뒤 비교해야 합니다. 실습 4의 코드가 `round(4)`를 쓴 이유가 여기 있고, 실습 8의 주석이 이 함정을 명시적으로 경고하고 있습니다.

실수 비교는 == 대신 이렇게

더 안전한 방법 세 가지

① 고유값 개수로 판정 — 가장 확실

```
상수 = [c for c in df_4.columns if df_4[c].nunique() == 1]
```

② 아주 작은 값보다 작은지로 판정

```
상수 = std_raw[std_raw < 1e-10].index.tolist()
```

③ round로 자릿수를 맞춰 비교 — 실습 코드 방식

```
상수 = std_raw.round(4)[std_raw.round(4) == 0].index.tolist()
```

▶ 세 방법 모두 7개 센서 + setting_3 = 8개를 정확히 잡아냅니다

`nunique() == 1`이 가장 확실합니다. 표준편차는 계산 과정에서 오차가 끼지만, 고유값 개수는 정수 비교라 오차가 없습니다. 실무에서는 이 방법을 기본으로 쓰는 편이 안전합니다.

9-6. 데이터를 보는 세 가지 질문

① 얼마나 큰가
shape · columns

② 어떻게 생겼나
head · info

③ 정상인가
describe · isna

질문	도구	오늘 확인한 것
① 얼마나 큰가	<code>df.shape</code> <code>df.columns</code>	1116행 26열
② 어떻게 생겼나	<code>df.head()</code> <code>df.info()</code>	엔진번호 · 회차 · 설정3 · 센서21
③ 정상인가	<code>df.describe()</code> <code>df.isna()</code>	결측 0 · 상수 컬럼 8개

개념

"1116행 26열, 엔진 5대, 결측 없음, 상수 센서 7개" — 이렇게 한두 줄로 메모를 남기는 것이 분석을 잘하는 사람의 공통점입니다. 값부터 들여다보면 길을 잃기 쉽습니다. 크기 → 구조 → 상태 순으로 좁혀 가는 것이 데이터 분석가의 첫 행동 습관입니다.

여기까지가 오늘 진도입니다. 실습 5번부터는 엔진 하나를 골라 정렬하고 추세를 읽는 단계인데, 다음 시간에 이어서 합니다. 오늘은 데이터가 어떻게 생겼는지 파악하는 것까지가 목표였습니다.

치트시트

오늘 배운 것을 한 장으로

A-1. Seaborn 그래프 여섯 개

묻고 싶은 것	함수	최소 코드	필요한 값
값이 어디에 몰렸나	histplot	<code>sns.histplot(data=df, x="온도")</code>	숫자 1개
요약과 이상치는	boxplot	<code>sns.boxplot(data=df, y="온도")</code>	숫자 1개
그룹별 개수는	countplot	<code>sns.countplot(data=df, x="라인")</code>	범주 1개
그룹별 평균은	barplot	<code>sns.barplot(data=df, x="라인", y="온도")</code>	범주+숫자
두 값이 연동되나	scatterplot	<code>sns.scatterplot(data=df, x="A", y="B")</code>	숫자 2개
여러 값의 관계는	heatmap	<code>sns.heatmap(df.corr(numeric_only=True))</code>	상관행렬

A-2. 공통 옵션

옵션	값	의미	쓰는 곳
data=	df	데이터프레임 통째로	모든 함수
x=	컬럼 이름 문자열	가로축	대부분
y=	컬럼 이름 문자열	세로축	boxplot 단일 · barplot · scatterplot
hue=	범주 컬럼 이름	색으로 그룹 나누기	거의 전부
palette=	"Set2" · "pastel" · "Reds_r"	색 조합	hue=와 함께만
legend=	False	범례 끄기	hue가 x와 같을 때
alpha=	0 ~ 1	투명도	scatterplot
bins=	정수 또는 리스트	구간 수	histplot
kde=	True	분포 곡선 추가	histplot
ax=	axs[0]	어느 칸에 그릴지	subplots와 함께

A-3. heatmap 완성형 네 옵션

네 개를 한 세트로 외우십시오

```
corr = df.corr(numeric_only=True)

plt.figure(figsize=(8, 6))
sns.heatmap(corr,
            annot=True,          # 각 칸에 숫자 표시
            fmt=".2f",          # 소수점 두 자리
            cmap="coolwarm",    # 빨강=양 · 파랑=음
            center=0)          # 0을 색의 한가운데로

plt.title("센서 상관관계")
plt.show()
```

A-4. 그래프 하나를 완성하는 표준 순서

이 순서를 외우면 대부분의 그래프를 만들 수 있습니다

```
# ① 준비 — 그날 한 번만
import pandas as pd
import seaborn as sns
import matplotlib.pyplot as plt
sns.set_theme(style="whitegrid")
plt.rcParams["font.family"] = "Malgun Gothic"
plt.rcParams["axes.unicode_minus"] = False

# ② 크기 지정 — 그리기 전에
plt.figure(figsize=(10, 6))

# ③ 본체 — sns
sns.histplot(data=df, x="소입로온도", hue="판정", bins=20, kde=True)

# ④ 설명 — plt
plt.title("판정별 소입로온도 분포")
plt.xlabel("소입로온도(°C)")
plt.ylabel("건수")

# ⑤ 저장 → 화면 — savefig가 show보다 먼저
plt.savefig("분포.png", dpi=150, bbox_inches="tight")
plt.show()
```

A-5. 시계열 용어와 도구

용어	뜻	오늘 데이터에서	다음 시간 도구
시계열	시간 순서로 기록되고 시간 정보가 붙은 데이터	C-MAPSS 1116행	—
timestamp	실제 날짜·시각이 시간 축	다음 시간 샘플	to_datetime
cycle	몇 번째 작동인가가 시간 축	time_cycles	sort_values
추세	값이 가는 방향	다음 시간 실습 5~6	rolling
변동성	값이 흔들리는 정도	주간 라인 std 2.39°C	rolling().std()
이상 징후	평소를 벗어난 튜	이상 12건	quantile
개체차	같은 모델인데 수명이 다름	179 ~ 287 회차	—

오류·함정 사전

X 이렇게 하면 → ✓ 바로잡기

B-1. 노트북 코드 교정 두 가지

제출된 실습 노트북에서 오류는 안 나지만 결과가 의도와 다른 부분이 두 군데 있었습니다.

① 18_01 실습 4 — 범주형 컬럼이 반복문에 섞여 들어감

무슨 일이 일어나나

```
# x cols[0]은 "라인" - 범주형입니다
cols = list(df_4.columns)
for c in cols[0:-1]:
    sns.histplot(data=df_4, x=c, hue="판정", bins=30)
    plt.show()
```

- ▶ 첫 번째 그림이 분포가 아니라 라인별 개수 막대 6개로 나옵니다
- ▶ 오류는 나지 않습니다

바로잡기

```
# ✓ 방법 1 - 인덱스를 하나 옮기기
for c in cols[1:-1]:
    sns.histplot(data=df_4, x=c, hue="판정", bins=30)
    plt.show()

# ✓ 방법 2 - 숫자형만 자동으로 고르기 (더 안전)
for c in df_4.select_dtypes("number").columns:
    sns.histplot(data=df_4, x=c, hue="판정", bins=30)
    plt.title(f"{c} - 판정별 분포")
    plt.show()
```

개념

`select_dtypes("number")`를 쓰면 컬럼 순서가 바뀌어도 안전합니다. 인덱스로 자르는 방식은 컬럼이 추가되거나 순서가 바뀌면 바로 깨집니다. 위치가 아니라 성질로 고르는 것이 좋은 습관입니다.

② 18_01 실습 6 — 필터한 변수를 쓰지 않음

무슨 일이 일어나나

```
# x day를 만들어 놓고 정작 df_6 전체를 그림
day = df_6[df_6["라인"] == "주간"]

sns.boxplot(data=df_6, x="라인", y="소입로온도", hue="판정")
plt.title("주간 라인 소입로 온도 IQR")
```

▶ 제목은 "주간 라인"인데 그림에는 주간·야간·특근 세 라인이 다 나옵니다

바로잡기

```
# ✓ 방법 1 — 정말 주간만 보려면 day를 쓰기
sns.boxplot(data=day, x="판정", y="소입로온도")
plt.title("주간 라인 소입로온도 IQR")

# ✓ 방법 2 — 세 라인 비교가 목적이면 필터를 지우고 제목을 고치기
sns.boxplot(data=df_6, x="라인", y="소입로온도", hue="판정")
plt.title("라인 x 판정 소입로온도 IQR")
```

주의

변수를 만들어 놓고 안 쓰는 것은 오류가 아니라 논리 버그라 더 위험합니다. 그림은 멀쩡히 나오고 제목만 틀리므로, 리포트에 넣으면 읽는 사람이 잘못된 결론에 도달합니다. 실행 후에는 항상 제목과 그림이 일치하는지 확인하십시오.

B-2. 자주 만나는 오류 열 가지

x 이렇게 하면	무슨 일이	✓ 바로잡기
<code>import matplotlib as plt</code>	<code>plt.title</code> 없음 — <code>AttributeError</code>	<code>import matplotlib.pyplot as plt</code>
<code>sns.histplot(df["온도"])</code>	동작은 하나 축 이름이 사라짐	<code>sns.histplot(data=df, x="온도")</code>
<code>sns.histplot(data=df, x="라인")</code>	범주형이라 개수 막대가 나옴	x에는 숫자형 컬럼만
<code>sns.boxplot(data=df, x="온도")</code>	상자가 가로로 누움	단일 분포는 y=
<code>sns.countplot(...,</code>	개수 그래프에 값은 불필요	<code>countplot</code> 은 x만

X 이렇게 하면	무슨 일이	✓ 바로잡기
y="온도")		
sns.heatmap(df)	원본 297행이 그대로 색칠됨	sns.heatmap(df.corr(...))
palette="Set2"	FutureWarning — 0.14에서 제거	hue="라인", legend=False
font 설정 뒤 set_theme	테마가 폰트를 덮어서 한글 깨짐	set_theme 먼저, 폰트 나중
show 뒤 savefig	빈 이미지 파일이 저장됨	savefig → show 순서
plt.title()	ax=로 그린 그래프에는 안 붙음	axs[0].set_title()

B-3. 그림은 맞는데 결론이 틀리는 경우

함정	왜 위험한가	대응
평균만 보고 그룹 비교	이상 856.88 vs 정상 859.53 — 평균은 거의 같음	표준편차와 범위를 함께 (4.9배 차이)
개수만 보고 비율 무시	그룹 크기가 다르면 결론이 뒤집힘	반드시 비율까지 계산
bins 하나만 보기	우연한 요철을 패턴으로 착각	여러 값으로 그려 확인
center=0 없이 heatmap	약한 음수도 진하게 보임	center=0 필수
박스플롯 점 = 불량	온도 32건 중 실제 불량은 12건	후보일 뿐 판정이 아님
상관으로 인과 주장	-0.736은 제어기 때문	공정 지식으로 해석
시계열을 순서 없이 봄	"올라가는 중"이 사라짐	그리기 전 sort_values
여러 엔진을 한 그래프에	다섯 흐름이 뒤엉켜 아무것도 안 보임	반드시 한 엔진씩

개념

B-2는 오류가 나는 실수라 금방 알아챱니다. B-3은 그림이 멀쩡하게 그려지는데 결론만 틀리는 실수라 훨씬 위험합니다. 오늘 데이터가 좋은 예입니다. 판정별 평균 온도만 보면 "이상이 오히려 낮다"는 이상한 결론이 나오지만, 표준편차를 보면 4.9배 차이이고 범위를 보면 이상 12건이 정상 띠 밖에 100% 있습니다. 그림 하나로 결론 내지 마십시오.

B-4. 부동소수점 비교

컴퓨터가 실수를 근사 저장하기 때문

```
# x 실수를 == 로 비교
print(0.1 + 0.2 == 0.3)
std = df.std(numeric_only=True)
print(len(std[std == 0]), "개")    # 상수 8개인데
```

- ▶ False
- ▶ 4 개

바로잡기

```
# ✓ 방법 1 - 고유값 개수로 (가장 확실)
상수 = [c for c in df.columns if df[c].nunique() == 1]
```

```
# ✓ 방법 2 - 아주 작은 값보다 작은지
상수 = std[std < 1e-10].index.tolist()
```

```
# ✓ 방법 3 - round로 자릿수를 맞춰
상수 = std.round(4)[std.round(4) == 0].index.tolist()
```

▶ 세 방법 모두 8개를 정확히 잡아냅니다

주의

실수(float)는 절대 ==로 비교하지 마십시오. 값이 논리적으로 같아도 계산 경로가 다르면 마지막 자리가 달라집니다. 개수·인덱스 같은 정수는 ==로 비교해도 안전합니다. 이 구분만 알아도 원인 모를 버그 상당수를 피할 수 있습니다.

실습 하이라이트와 다음 시간

오늘의 결론 · 예고

C-1. 실습 한눈에

교안	실습	한 일	핵심 결과
18_01	1	소입로온도 histplot	평균 859.42°C · 표준편차 1.998
	2	bins · kde 조절	bins에 따라 모양이 달라짐
	3	hue="판정" 겹쳐 보기	이상 std 6.91 vs 정상 1.42
	4	반복문으로 다섯 센서	판정별 분포 다섯 장
	5	소입로온도 boxplot	IQR 1.4°C · 바깥 점 32건
	6	라인별 · 라인×판정	주간 std 2.39 로 최대
	7	countplot · barplot	라인 100 / 99 / 98
	8	라인별 불량률	주간 7.0% · 야간 3.0% · 특근 2.0%
18_02	1	산점도 두 쌍	온도-출력 -0.736 · 온도-CP -0.026
	2	hue로 위험 영역	이상 12건 = 정상 띠 밖 100%
	3	상관 heatmap	10쌍 중 뚜렷한 것 1쌍뿐
	4~6	subplots 리포트	2×2 리포트 저장
20_01	1	C-MAPSS 구조	1116행 26열
	2	엔진 단위	5대 · 179 ~ 287 회차
	3	센서 범위	평균 641.75 / 47.69 / 517.43
	4	쓸 센서 고르기	상수 8개 제외 → 16개 사용

C-2. 오늘 데이터가 말해 준 것

열여섯 개의 실습을 흩어진 조각이 아니라 **하나의 조사 과정**으로 다시 읽어 보겠습니다.

설비 적용

① **평균 비교가 실패했습니다.** 판정별 평균 온도는 이상 856.88°C, 정상 859.53°C로 오히려 이상이 낮습니다.

평균만 보면 "저온이 불량 원인"이라는 엉뚱한 결론이 나옵니다.

② **히스토그램이 첫 실마리를 줬습니다.** hue="판정"으로 겹쳐 보니 빨간 조각이 파란 봉우리의 **양쪽 바깥**에 있었습니다. 표준편차를 재 보니 이상 6.91 대 정상 1.42로 **4.9배** 차이였습니다.

③ **범위로 확정했습니다.** 정상의 온도 범위는 855.3~865.4°C인데, **이 안의 285건 중 이상은 0건, 밖의 12건은 전부 이상**이었습니다. 위험 영역은 "높은 온도"가 아니라 **정상 띠의 양쪽 바깥**입니다.

④ **산점도가 원인을 보여줬습니다.** 상관 -0.736의 우하향 띠에서 이상 12건의 위치를 보니, 저온 9건은 제어출력 **95.7~100%**, 고온 3건은 **0.6~5.6%**였습니다. 중간은 하나도 없습니다. **제어기가 한계까지 밀린 포화 상태**입니다.

⑤ **박스플롯이 어디를 볼지 알려줬습니다.** 주간 라인의 표준편차 2.39°C가 가장 크고, 불량률도 7.0%로 가장 높습니다. **변동이 큰 라인에서 불량률이 많다** — 두 경로가 같은 결론에 도달했습니다.

그리고 이 다섯 단계를 만드는 데 쓴 것은 **Seaborn 함수 여섯 개**가 전부입니다. 나머지는 21~24일차에 배운 pandas 입니다. **한 줄로 그릴 수 있게 되니 여덟 장을 그려 보게 되었고, 그래서 평균만 봐서는 절대 못 찾았을 사실을 찾았습니다.**

C-3. 21~26일차 여섯 회차의 연결



회차	배운 것	오늘 어디서 다시 쓰었나
21일차	<code>groupby()</code> · <code>corr()</code>	실습 7·8 라인별 집계 · 실습 3 heatmap
22일차	<code>isna().sum()</code>	C-MAPSS 결측 0 확인
23일차	<code>fillna()</code> · 대체의 부작용	오늘 데이터는 결측 0이라 불필요
24일차	<code>quantile()</code> · IQR 경계	boxplot이 자동 계산 — 경계 856.7~862.3
25일차	<code>hist</code> · <code>bar</code> · <code>scatter</code> · <code>subplot</code>	같은 그래프를 한 줄로
26일차	<code>hue</code> · <code>heatmap</code> · <code>time_cycles</code>	그룹 비교와 시계열의 시작

연결

오늘 새로 배운 Seaborn 함수는 **여섯 개**뿐이고, 시계열은 아직 **개념과 구조 파악**까지입니다. 나머지는 전부 21~25일차의 반복입니다. **새 도구가 늘어난 것이 아니라, 같은 도구를 더 빨리 쓰게 된 것입니다.** 그리고 그 속도가 발견의 양을 바꿉니다.

C-4. 다음 시간 예고

강사님 말씀대로 **20_01 실습 5번부터 이어서** 시작합니다. 20장을 마무리하고 21장 세 파트로 넘어갈 예정입니다.

순서	내용	핵심 도구	무엇을 하나
① 20_01 실습 5~8	엔진 추세 읽기	<code>sort_values</code> <code>iloc</code>	첫·끝 값으로 상승·하강 판정
② 20_02	시간 인덱스와 집계	<code>to_datetime</code> <code>resample</code>	timestamp 데이터로 전환
③ 21_01~03	추세·변동성·다중 센서	<code>rolling diff</code>	이동평균과 변화량

개념

오늘 STEP 8에서 배운 **timestamp**와 **cycle**을 오가는 학습 순서가 그대로 이어집니다. cycle 데이터인 C-MAPSS로 구조를 익혔으니(오늘), 다음은 timestamp 샘플로 **시·일 단위 집계**를 배우고(20_02), 그다음 다시 cycle로 돌아와 **이동평균과 변화량**을 다룹니다(21_01~03). 두 시간 축을 오가며 **어떤 시계열에도 알맞은 도구를 고르는 힘**을 기르는 구성입니다.

미리보기

```
# 다음 시간 실습 5의 첫 줄 — 오늘 배운 것 위에 얹힙니다
engine_5 = df_5[df_5["unit_nr"] == 2].sort_values("time_cycles")
```

```
# 왜 sort_values가 필요한가
# 시계열은 순서가 생명이라 그리기 전에 반드시 정렬합니다
start = engine_5["s_11"].iloc[0]
end = engine_5["s_11"].iloc[-1]
print("상승" if end > start else "하강")
```

▶ 오늘 실습 2에서 확인한 "엔진 5대가 섞여 있다"를 알아야 이 줄이 이해됩니다

오늘의 성취와 복습 루틴

오늘 할 수 있게 된 것

아래 여섯 가지를 코드를 보지 않고 할 수 있다면 오늘 수업은 소화한 것입니다.

#	할 수 있게 된 것	확인 방법
1	Seaborn 준비 코드를 외워서 쓴다	<code>set_theme</code> 다음에 폰트 — 순서를 아나
2	여섯 그래프를 목적에 맞게 고른다	"그룹별 평균" → <code>barplot</code> 이 바로 떠오르나
3	<code>hue</code> 로 그룹을 나눠 비교한다	딕셔너리 없이 색을 나눌 수 있나
4	상관행렬을 heatmap으로 읽는다	<code>center=0</code> 이 왜 필요한지 설명할 수 있나
5	<code>subplots</code> 로 리포트를 만든다	<code>ax=</code> 와 <code>set_title</code> 의 관계를 아나
6	시계열과 아닌 것을 구별한다	"순서를 섞으면?"으로 판정할 수 있나

복습 루틴 — 3일 계획

시점	할 일	시간
오늘 저녁	18_열처리.csv로 여섯 그래프를 코드 안 보고 다시 그려 보기	30분
내일	부록 A-4 표준 순서를 손으로 써 보고, <code>savefig</code> 위치가 맞는지 대조	15분
주말	20_cmapss 데이터로 2번이 아닌 다른 엔진을 골라 실습 1~4 반복	40분

주의

Seaborn은 코드가 짧아서 다 아는 것 같은 착각을 주기 쉽습니다. 빈 셀에서 시작하면 `data=`를 빼먹거나 `plt.show()`를 잊습니다. 한 번 막혀 봐야 외워집니다.

오늘 배운 여섯 그래프는 앞으로 거의 모든 분석의 첫 단계가 됩니다. 그리고 후반부터는 순서가 있는 데이터라는 새로운 세계에 들어섰습니다. 그 도구는 다음 시간부터 배웁니다.

수고하셨습니다.